ZSCRIPT

The Zscript Book

Write your own chart indicators, from your first plot to order book and options data.

TradeZoro · Education only. Not financial advice.

The Zscript Book. © TradeZoro. All rights reserved.

Education only. Not financial advice. Every script in this book is a teaching sample, not a trading result, and none of it promises profit. Data features depend on your plan.

If you know other chart scripting languages, Zscript will feel familiar.

Contents

Part 1. The language

Getting started 8
Anatomy of a script 11
Bars and series 14
Types and values 18
Decisions and loops 22
Functions 25
Inputs 27
Plots and drawings 29
Alerts 33
Our data 35
Publishing and selling 40
Limits 42
Errors explained 45
Error and warning codes 49
Patterns every scripter uses 54
Build four indicators step by step 58
Glossary 63

Part 2. Reference

General 66
Declaration and inputs 70
Indicators (ta) 83
Math 110
Text (str) 119

Colors 126
Arrays 129
Maps 144
Plots and chart colors 148
Labels 158
Lines 166
Boxes 175
Tables 183
Alerts 187
Other symbols and timeframes 188
Order book (book) 189
Order flow (flow) 192
GEX levels (gex) 195
ZoroOptions (opt) 197
Built-in variables 199
Named constants 208

Part 3. Cookbook

Simple moving average 213
EMA ribbon 214
Average cross marks 215
RSI with zones 216
MACD with colored histogram 217
Bollinger Bands with shading 218
Keltner channel 219
ATR trailing line 220
Supertrend 221

Session VWAP 222
Stochastic 223
CCI 224
On-balance volume 225
Money flow index 226
Donchian channel 227
Swing high and low labels 228
Previous day high and low 230
Higher timeframe trend 231
Inside bars 232
Engulfing candles 233
Fair value gaps 234
Opening range box 235
Session background 237
Volume spike marks 238
Relative volume 239
Bar range as percent 240
Opening gap marks 241
Label the day's high 242
Info table 243
Gradient volume 244
Trend filter background 245
Bars since last cross 246
Correlation with another symbol 247
Regression line and bands 248
Order book imbalance 249
Nearest book walls 250

Delta columns ... 251
Cumulative delta ... 252
Delta against price ... 253
Big trade count ... 254
POC and value area ... 255
GEX levels ... 256
Above or below the flip ... 257
Distance to the call wall ... 258
Option range edges ... 259
IV rank readout ... 260
Gamma nodes with GEX walls ... 261
Cross alert ... 262
Level alert with a message ... 263

Part 4. The course

Lesson 1. Meet Zscript ... 265
Lesson 2. Bars and series ... 267
Lesson 3. Values, types and colors ... 268
Lesson 4. Moving averages and indicators ... 269
Lesson 5. Decisions and signals ... 270
Lesson 6. Remembering with var ... 271
Lesson 7. Inputs ... 272
Lesson 8. Loops, arrays and functions ... 273
Lesson 9. Drawing: labels, lines, boxes and tables ... 274
Lesson 10. Our data: order book, order flow, GEX and options ... 275
Lesson 11. Alerts and request.security ... 276
Lesson 12. Limits, errors and publishing ... 277

Part 1. The language

Getting started

Zscript is the small language you use on TradeZoro to build your own chart indicators. You write a few lines, press **Add to chart**, and your idea is drawn on the chart. It is free for everyone: writing, saving and publishing scripts.

If you know other chart scripting languages, Zscript will feel familiar. If you have never written a line of code, that is fine too. This manual starts at zero.

What you can build

- Lines and histograms from price, such as moving averages and momentum.
- Marks on the chart, such as arrows when two lines cross.
- Colored backgrounds, boxes, lines and labels.
- Tables with numbers you care about.
- Alerts that fire when your rule is true.
- Indicators that use data most scripting languages do not have: the order book, order flow, GEX levels and ZoroOptions numbers. See Our data.

Your first script

Open ZoroCharts, open the script editor panel, and choose **Zscript** as the language. Paste this:

```
//@zscript 1
indicator("My first line", overlay=true)

average = ta.sma(close, 20)
plot(average, title="20-bar average", color=color.new(#03c988, 0),
linewidth=2)
```

Press **Add to chart**. A green line appears on the price chart. It is the average of the last 20 closing prices, recalculated on every bar.

Here is what each line does:

1. `//@zscript 1` tells the editor this is a Zscript file, version 1. Always the first line.
2. `indicator(...)` names your script. `overlay=true` draws it on top of the price candles. Leave it out (or use `false`) to get a separate pane under the chart.
3. `average = ta.sma(close, 20)` makes a value called `average`. `ta.sma` is a built-in function: the simple moving average. `close` is the closing price of each bar.
4. `plot(...)` draws the value as a line.

Change something

Edit the number `20` to `50` and press **Add to chart** again. The line gets smoother and slower. That is the whole loop: change, run, look.

Where scripts live

Your scripts are saved in your account, next to your chart layout. You can keep as many as you like. Publishing one to the Script Library is optional and free; see Publishing and selling.

What to read next

- Your first script, line by line explains the shape of every script.
- Bars and series explains the one idea that makes everything else easy.
- The Cookbook has working scripts you can copy and change.

Anatomy of a script

Every script has the same four parts, always in this order.

1. The **header**: `//@zscript 1`.
2. The **declaration**: one `indicator(...)` call.
3. The **inputs**: settings the person using your script can change.
4. The **logic and drawing**: calculations, then `plot` and friends.

```
//@zscript 1
indicator("Anatomy demo", overlay=false)

length = input.int(14, "Length", minval=2)
momentum = ta.rsi(close, length)

plot(momentum, "RSI")
hline(70)
hline(30)
```

Comments

Anything after `//` on a line is a comment. The computer ignores it. Use comments to remind yourself why, not what.

```
// this line is ignored
x = close // so is this part
```

Names

Names of values may use letters, digits and `_`, and cannot start with a digit. Names are case sensitive: `Length` and `length` are two different things. Pick names a stranger could read: `fastLength` beats `a1`.

Lines and blocks

One statement per line. Code that belongs together is indented under its header, with two or four spaces. The indent is how Zscript knows where a block ends.

```
up = close > open
bgcolor(up ? color.new(#03c988, 85) : na)
```

A long line can break after an operator or a comma. Indent the continuation by at least one more level than the line it continues.

Running order

Your script is not run once. It is run **once for every bar** on the chart, from the oldest bar to the newest, and again on every tick of the newest bar. Every line sees the world as it was at that bar. The next chapter shows why this matters.

The declaration

`indicator()` takes a title and a few optional settings.

Setting	What it does
`title`	The name shown on the chart and in the list.
`overlay`	`true` draws on the price chart. `false` (default) uses its own pane.
`max_bars_back`	How far back your script may look. Leave it alone unless you see an error asking for it.

A script has exactly one `indicator()` call, and it must come before any drawing.

Bars and series

One run per bar

A chart is a row of bars. Zscript runs your whole script for each bar. When it is working on bar number 500, `close` means "the close of bar 500". When it moves on to bar 501, `close` now means that bar's close.

A value that changes bar by bar is called a **series**. `open`, `high`, `low`, `close`, `volume` and `time` are series that exist in every script. Anything you calculate from a series is a series too.

Looking back

To see an older value of a series, put a number in square brackets after it. `close[1]` is the close one bar ago, `close[2]` is two bars ago, and so on.

```
//@zscript 1
indicator("Bar change")

change1 = close - close[1]
plot(change1, "Change since last bar", style=plot.style_histogram)
```

On the very first bar there is no bar before it, so `close[1]` has no value. Zscript calls "no value" **na** (not available). Any math with `na` gives `na`, and a plot skips `na` bars. That is by design: no value means no line, not a wrong line.

You can test for it and fix it:

```
prev = nz(close[1], close)   // use close itself when there is no
older bar
isMissing = na(close[1])
```

`nz(x, replacement)` swaps `na` for the replacement (or 0 if you leave it out). `na(x)` is `true` when x is missing.

Remembering across bars with var

An ordinary value is recalculated from scratch on each bar. To keep a number alive from bar to bar, declare it with `var`. A `var` is set once, on the first bar, and keeps whatever you assign to it after that.

```
//@zscript 1
indicator("Count green bars")

var int greens = 0
if close > open
    greens := greens + 1

plot(greens, "Green bars so far")
```

Two things to notice. First, `:=` changes an existing value; a single `=` creates a new one. Second, `greens` never resets, so the plot only goes up.

Realtime bars: var and varip

On historical bars each bar runs once. The newest bar runs again on every tick. A plain `var` is rolled back to its value from the end of the last finished bar before each tick, so history and live look the same. Use `varip` only when you truly need a value to survive between ticks of the same bar, for example to count ticks.

```
//@zscript 1
indicator("Ticks on this bar")

varip int ticks = 0
if barstate.isnew
    ticks := 0
ticks := ticks + 1
plot(ticks)
```

On finished history bars this plots 1, because each bar runs once. On the live bar it counts up.

Series are not arrays

A series is "one value per bar". An **array** is a list you fill and read yourself, on a single bar. Both exist; see Types and values.

Rules of thumb

- Think in terms of "what is true on this bar".
- Use `[n]` to look at the past, never the future. Zscript has no way to read a bar that has not happened yet. That is on purpose, and the same rule holds for order book, order flow, GEX and options numbers: you only see what was known at that bar.

- Use `var` when something must be remembered.

Types and values

Every value has a type. Zscript works out the type for you most of the time, and you can write it when you want to be clear.

Type	Holds	Example
`int`	whole numbers	`14`
`float`	numbers with a decimal part	`0.5`
`bool`	`true` or `false`	`close > open`
`string`	text	`"Buy zone"`
`color`	a color	`#03c988`, `color.red`
`series<T>`	one value of type T per bar	`close`
`array<T>`	a list of values of type T	`array.new<float>()`
`map<K,V>`	a lookup table	`map.new<string, float>()`

Making and changing values

```
//@zscript 1
indicator("Values")

float range1 = high - low        // type written out
mid = (high + low) / 2           // type worked out
bool bigBar = range1 > ta.sma(range1, 20) * 1.5

var float biggest = 0.0
if range1 > biggest
    biggest := range1

plot(mid, "Mid")
plotshape(bigBar, "Big bar", shape.circle, location.abovebar)
```

- `=` creates a value. `:=` changes one that already exists.
- Math: `+ - * / %`. Compare with `== != < <= > >=`. Combine tests with `and`, `or`, `not`.
- Text joins with `+`: `"Range " + str.tostring(range1)`.
- `x ? a : b` gives `a` when `x` is true, else `b`.

Colors

Write a color as `#rrggbb`, use a named one such as `color.green`, or build one with `color.new(base, transparency)` where transparency is 0 (solid) to 100 (invisible). `color.rgb(r, g, b)` builds one from numbers.

Arrays

```
//@zscript 1
indicator("Array demo")

var prices = array.new<float>()
array.push(prices, close)
if array.size(prices) > 5
    array.shift(prices)

plot(array.avg(prices), "Average of the last 5 closes")
```

Maps

```
//@zscript 1
indicator("Map demo")

var counts = map.new<string, int>()
key = close > open ? "up" : "down"
map.put(counts, key, nz(map.get(counts, key)) + 1)
plot(nz(map.get(counts, "up")), "Up bars")
```

Tuples

A function can return several values at once. Take them apart with square brackets.

```
//@zscript 1
indicator("Tuple demo", overlay=true)

[middle, upper, lower] = ta.bb(close, 20, 2.0)
plot(middle)
plot(upper)
plot(lower)
```

Limits to know

Arrays, maps, series and drawings each have a size cap so a script cannot use all your memory. See Limits.

Decisions and loops

if / else

```
//@zscript 1
indicator("Trend color", overlay=true)

fast = ta.ema(close, 9)
slow = ta.ema(close, 21)

lineColor = color.gray
if fast > slow
    lineColor := color.green
else if fast < slow
    lineColor := color.red

plot(fast, color=lineColor, linewidth=2)
```

`if` can also give back a value:

```
label1 = if close > open
    "up"
else
    "down"
```

switch

Pick one branch out of several by value:

```
//@zscript 1
indicator("Pick an average", overlay=true)

kind = input.string("EMA", "Type", options=["SMA", "EMA", "WMA"])
len = input.int(20, "Length")

smaLine = ta.sma(close, len)
wmaLine = ta.wma(close, len)
emaLine = ta.ema(close, len)

line1 = switch kind
    "SMA" => smaLine
    "WMA" => wmaLine
    => emaLine

plot(line1)
```

The lone `=>` is the default branch. We work out all three averages first, on every bar, and let the `switch` only choose. Calling `ta` functions inside a branch would leave their memory with gaps.

for loops

`for` counts from one number to another. Use it to look at the last few bars one by one.

```
//@zscript 1
indicator("Count higher closes")

n = input.int(10, "Look at last", minval=1, maxval=50)
higher = 0
for i = 0 to n - 1
    if close[i] > close[i + 1]
        higher += 1

plot(higher, style=plot.style_columns)
```

`for x in myArray` walks the items of an array.

while

```
n = 0
while n < 10 and close[n] > open[n]
    n += 1
```

Keep loops short

Zscript runs your script on every bar, so a loop that runs 500 times does 500 times the work on every one of thousands of bars. Two safeguards protect you and everyone else:

- A loop has a maximum number of turns per bar, and your script has a maximum amount of work per bar. Hit either and the script stops with a clear message on the line that went too far.
- The first run must finish in 20 seconds.

When you can, use the built-in functions such as `ta.highest` or `ta.sma`: they are faster than the same logic in a loop. More in Limits.

`break` leaves a loop early. `continue` skips to the next turn.

Functions

One-line functions

```
//@zscript 1
indicator("Typical price band", overlay=true)

typical(h, l, c) => (h + l + c) / 3
band(x, pct) => x * (1 + pct / 100)

tp = typical(high, low, close)
plot(tp)
plot(band(tp, 1.0))
plot(band(tp, -1.0))
```

Functions with several lines

The last line is the value the function gives back.

```
//@zscript 1
indicator("Smoothed range", overlay=false)

smoothRange(len) =>
    r = high - low
    avg = ta.sma(r, len)
    avg * 100 / close

plot(smoothRange(14), "Range as % of price")
```

Returning more than one value

```
//@zscript 1
indicator("Two bands", overlay=true)

channel(len) =>
    top = ta.highest(high, len)
    bottom = ta.lowest(low, len)
    [top, bottom]

[t, b] = channel(20)
plot(t)
plot(b)
```

Things to know

- A function must be defined before the line that uses it.
- Functions see the series history of what you pass in: inside `f(x)`, `x[1]` is the previous bar's value of whatever you passed.
- A function that calls `ta.*` functions should be called on every bar, not only inside an `if`. The `ta` functions keep a running history, and skipping bars gives wrong answers. Zscript warns you when it spots this.
- Functions cannot call themselves.

Inputs

Anything you want to be changeable goes in an `input.*` call. Each one adds a control to your script's settings dialog.

Call	Control	Example
`input.int`	whole number box	`input.int(14, "Length", minval=1)`
`input.float`	decimal box	`input.float(2.0, "Multiplier", step=0.1)`
`input.bool`	switch	`input.bool(true, "Show labels")`
`input.string`	text or a pick list	`input.string("EMA", "Type", options=["SMA","EMA"])`
`input.color`	color picker	`input.color(#03c988, "Line color")`
`input.source`	pick a price (close, hl2...)	`input.source(close, "Source")`
`input.timeframe`	timeframe picker	`input.timeframe("60", "Higher timeframe")`

```
//@zscript 1
indicator("Tunable bands", overlay=true)

src    = input.source(close, "Source")
len    = input.int(20, "Length", minval=2, maxval=500,
group="Average")
mult   = input.float(2.0, "Width", minval=0.1, step=0.1,
group="Average")
shade  = input.bool(true, "Shade between bands", group="Look")
tint   = input.color(#00adb5, "Tint", group="Look")

[mid, up, lo] = ta.bb(src, len, mult)
p1 = plot(up, color=tint)
p2 = plot(lo, color=tint)
plot(mid, color=color.gray)
fill(p1, p2, color=shade ? color.new(tint, 90) : na)
```

Good habits

- Give every input a short, plain title and a sensible default. The default is what everyone sees first.
- Set `minval` and `maxval` so nobody can type 0 into a length.
- Use `group="..."` to keep long lists tidy.
- Add `tooltip="..."` to explain anything a newcomer would ask about.
- Inputs are read once per run. You cannot change an input while the script is running.

Plots and drawings

plot

`plot(series, title, color, linewidth, style)` draws one value per bar.

Styles: `plot.style_line` (default), `plot.style_histogram`, `plot.style_columns`, `plot.style_area`, `plot.style_circles`, `plot.style_stepline`, `plot.style_cross`.

```
//@zscript 1
indicator("Volume with color")

up = close >= open
plot(volume, "Volume", color=up ? color.green : color.red,
style=plot.style_columns)
```

A color can differ bar by bar, as above.

hline and fill

`hline(price)` draws a flat line across the pane. `fill(a, b, color)` shades between two plots or two hlines.

```
//@zscript 1
indicator("RSI zone")

r = ta.rsi(close, 14)
top = hline(70)
bot = hline(30)
fill(top, bot, color.new(color.gray, 90))
plot(r, "RSI")
```

Marks on bars

`plotshape(condition, title, shape, location)` puts a small shape on every bar where the condition is true. `plotchar` does the same with a letter or symbol. `plotarrow` draws an arrow whose length follows a number.

```
//@zscript 1
indicator("Cross marks", overlay=true)

fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast)
plot(slow)
plotshape(ta.crossover(fast, slow), "Cross up", shape.triangleup,
location.belowbar, color.green)
plotshape(ta.crossunder(fast, slow), "Cross down",
shape.triangledown, location.abovebar, color.red)
```

Coloring the chart itself

`bgcolor(color)` tints the pane background on a bar. `barcolor(color)` recolors the candle. `plotcandle(o, h, l, c)` draws your own candles.

```
//@zscript 1
indicator("Inside bars", overlay=true)

inside = high < high[1] and low > low[1]
barcolor(inside ? color.yellow : na)
bgcolor(inside ? color.new(color.yellow, 90) : na)
```

Labels, lines, boxes and tables

These are **drawings**. Unlike a plot, they are objects you create, move and delete. Each has a `.new` call and `set_` calls to change it.

```
//@zscript 1
indicator("Swing labels", overlay=true)

ph = ta.pivothigh(high, 5, 5)
pl = ta.pivotlow(low, 5, 5)
if not na(ph)
    label.new(bar_index - 5, ph, "H",
style=label.style_label_down, color=color.red,
textcolor=color.white)
if not na(pl)
    label.new(bar_index - 5, pl, "L", style=label.style_label_up,
color=color.green, textcolor=color.white)
```

```
//@zscript 1
indicator("Today's range box", overlay=true)

var box rangeBox = na
if dayofmonth != dayofmonth[1]
    rangeBox := box.new(bar_index, high, bar_index, low,
border_color=color.gray, bgcolor=color.new(color.gray, 90))
else
    box.set_right(rangeBox, bar_index)
    box.set_top(rangeBox, math.max(box.get_top(rangeBox), high))
    box.set_bottom(rangeBox, math.min(box.get_bottom(rangeBox),
low))
```

```
//@zscript 1
indicator("Info table", overlay=true)

var t = table.new(position.top_right, 2, 2)
if barstate.islast
    table.cell(t, 0, 0, "Close")
    table.cell(t, 1, 0, str.tostring(close, "#.##"))
    table.cell(t, 0, 1, "Volume")
    table.cell(t, 1, 1, str.tostring(volume, "#"))
```

`line.new(x1, y1, x2, y2)` draws a segment. `line.delete(id)` removes it. Each kind has `get_` and `set_` calls, and a `*.delete` call. The reference lists them all.

Drawing limits

Each kind of drawing has a maximum count on screen (labels, lines, boxes, tables). When you go over, the oldest ones are removed first. If you create drawings in a loop, delete the ones you no longer need. See Limits.

Alerts

An alert is a rule that is true or false on each bar. When it turns true, TradeZoro can tell you.

alertcondition

Declare the rule once. It then shows up in the alert dialog on the chart, where you choose how to be told.

```
//@zscript 1
indicator("Cross alerts", overlay=true)

fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast)
plot(slow)

alertcondition(ta.crossover(fast, slow), "Fast crossed above
slow", "9 EMA is above 21 EMA")
alertcondition(ta.crossunder(fast, slow), "Fast crossed below
slow", "9 EMA is below 21 EMA")
```

alert()

`alert(message)` fires from inside your code, which lets you build the message from values.

```
//@zscript 1
indicator("Big bar alert", overlay=true)

big = (high - low) > ta.atr(14) * 2
if big and barstate.isconfirmed
    alert("Big bar: range " + str.tostring(high - low, "#.##"))
```

`barstate.isconfirmed` is true once the bar has closed. Alerting only on closed bars avoids alerts that appear and vanish within a bar.

Good to know

- Alerts run while the chart is open in your browser, using the same data as the chart.
- A message is plain text. Keep it short.
- An alert on data from the order book, order flow, GEX or options follows the same plan rules as showing that data on a chart.

Our data

Most scripting languages stop at price and volume. Zscript can also read the extra data TradeZoro already shows on its charts. Each kind lives in its own namespace.

Namespace	What it is	Plan
`book.*`	Order book (level 2): resting orders above and below price	Pro
`flow.*`	Order flow: trades, delta, big trades, footprint, volume profile	Pro
`gex.*`	GEX+ levels: call wall, put wall, flip, zero gamma	GEX+
`opt.*`	ZoroOptions summary: gamma nodes, ranges, IV rank, max pain	Pro

Your script gets the data your plan gives you on the chart, no more. When your plan does not include a data kind, the call returns `na` and you see one short notice on the chart, once. The script keeps running; the rest of your lines still draw.

No peeking

Every value is the value known **at that bar**. There is no way to read a future bar, so a script that uses this data looks the same on a chart and in Replay. Order book, GEX and options numbers are snapshots: a snapshot is stored at some time and used for the bars that follow it, until the next one arrives. For GEX and ZoroOptions numbers, treat them as "updated a few minutes ago", not as a tick-by-tick feed.

Order book: book

```
book.bid(level)        price of the bid at depth `level` (0 = best)
book.ask(level)        price of the ask at depth `level`
book.bidsize(level)    size resting there
book.asksize(level)    size resting there
book.depth(side, n)    total size in the best n levels of a side
(book.side_bid or book.side_ask)
book.imbalance(n)      (bid depth - ask depth) / (bid depth + ask
depth) in the best n levels, -1 to 1
book.wall(side, minSize)  price of the nearest level with at least
minSize resting (side is book.side_bid or book.side_ask)
book.heat(price)       size resting at that price (nearest level
within one tick)
```

Available where we have a depth feed: the futures hub and the supported crypto venues. On other symbols you get `na` and a notice.

```
//@zscript 1
indicator("Book imbalance", overlay=false)

imb = book.imbalance(10)
plot(imb, "Imbalance (10 levels)", style=plot.style_columns,
color=imb >= 0 ? color.green : color.red)
hline(0)
```

Order flow: flow

```
flow.buyvol, flow.sellvol    volume that traded at the ask / at the
bid, per bar
flow.delta                   buyvol - sellvol for the bar
flow.cvd                     running total of delta
flow.trades                  number of trades in the bar
flow.bigtrades(minSize)      count of single trades of at least
minSize
flow.footprint(price)        volume traded at one price level
inside the bar
flow.bigtrades_vol(minSize)  total size of those big trades
flow.poc, flow.vah, flow.val point of control, value area high and
low of the bar
```

```
//@zscript 1
indicator("Delta and CVD")

plot(flow.delta, "Delta", style=plot.style_columns,
color=flow.delta >= 0 ? color.green : color.red)
plot(flow.cvd, "CVD", color=color.white)
```

GEX levels: gex

```
gex.callwall   strike with the largest call gamma
gex.putwall    strike with the largest put gamma
gex.flip       gamma flip level
gex.zero_upper, gex.zero_lower   the two zero-gamma levels around
price
gex.net        net gamma number
gex.level(n)   the nth strongest level (0 = strongest)
gex.updated    when the levels were computed (ms)
```

ES and NQ use the same mapping as the GEX+ page.

```
//@zscript 1
indicator("GEX walls", overlay=true)

plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
plot(gex.flip, "Flip", color=color.yellow,
style=plot.style_stepline)
```

ZoroOptions: opt

```
opt.node(n)      price of the nth gamma node
opt.range(kind)  a pair [low, high] for kind "day", "week" or
"expected"
opt.ivrank       IV rank, 0 to 100
opt.pc           put/call ratio
opt.maxpain      max pain strike
opt.zone         the current zone label
opt.updated      when the data was computed (ms): show it as
"updated X min ago"
```

```
//@zscript 1
indicator("Option ranges", overlay=true)

[lo, hi] = opt.range("expected")
plot(hi, "Range high", color=color.gray, style=plot.style_stepline)
plot(lo, "Range low", color=color.gray, style=plot.style_stepline)
```

Handling "no data"

Because these calls return `na` when data is missing, protect your math with `nz` or `na()`:

```
//@zscript 1
indicator("Safe delta", overlay=false)

d = nz(flow.delta, 0)
plot(d, "Delta (0 when no data)", style=plot.style_columns)
```

Education only. Data samples in this manual are labelled "Sample" and are not results.

Publishing and selling

Publishing is free for everyone. You can share a script openly, or sell it through Creator Hub.

Publish a script

1. Open your script in the editor and press **Publish**.
2. Give it a name and a short description. Pick a category.
3. Choose how people get it: - **Open source**: anyone who adds it can read the code. - **Protected**: people can use it but not read it.
4. Choose **free** or **paid**.
5. Add a short note about what changed in this version.

The script then appears in the Script Library. Each time you publish again, you add a new version. People who already use your script can update when they choose.

Protected scripts

When you publish a protected script, the source never leaves your account. We compile it, seal the compiled program on our server, and only send it to people who are allowed to use it. Paid and protected scripts run only for people who have it. Nobody else is sent the code.

A protected script runs in the same sandbox as every other script. It cannot reach the network, your storage or the page.

Selling

A paid script is listed in Creator Hub like other creator products. Payouts work the same way as the rest of Creator Hub; see the Creator Hub rules for the payout terms. We do not publish prices in this manual: you set yours in the publish dialog.

Rules for listings

- Describe honestly what the script does.
- Do not claim profit, win rates or results. Education only. Not financial advice.
- Sample screenshots must say "Sample".
- A script that tries to break the sandbox, hide what it does, or harm the viewer is removed.

Data in published scripts

If your script uses book, flow, GEX or options data, people who run it see that data only if their plan includes it, exactly as on a chart. Say so in your description so nobody is surprised.

Limits

Scripts from other members run in your browser. To make that safe, every script runs inside a locked room: it can do math and draw on the chart, and nothing else. It cannot open web connections, read your cookies or storage, or touch the page.

What a script can never do

- Reach the network.
- Read or write your browser storage or cookies.
- Change the page, other than through the plots and drawings it declares.
- Run text as code. Zscript has no `eval`.

The numbers

These are the limits the engine enforces today.

What	Limit
Script size	256 KB, 20,000 lines
Nesting depth	128
Bars in one run	50,000
First full run	20 seconds
Work per bar	1,000,000 operations
Turns of one loop, per bar	100,000
Function calls inside each other	64 (a function cannot call itself)
Items in one array	100,000 (1,000,000 across all arrays)
Keys in one map	50,000
Length of one string	4,096 characters
Plots, marks, fills, hlines, bgcolor and barcolor calls	64 together
Labels, lines, boxes	50 each by default, up to 500 with `max_labels_count` and friends
Tables	up to 20; 1,000 cells each
Alerts kept per run	500
`request.security` calls	16
Look-back with `[n]`	500 bars by default, up to 5,000 with `max_bars_back`

What happens when you hit one

Limit	What you see
Run takes longer than 20 seconds	The run stops (E401).
Too much work on one bar	The script stops on that bar and shows the line (E403).
A loop turns too many times	The loop stops with an error on its line (E404).
Too many items in an array or map	Error on the line that added the extra item (E405, E406).
Too many plots	Error when the script is added (E341).
Too many labels, lines or boxes	The oldest are removed first. This is not an error.
Too many tables	Error (E408).

Every code is explained in Error and warning codes.

Staying under the limits

- Prefer built-ins (`ta.highest` , `ta.sma`) over loops.
- Keep loops short and fixed in length.
- Delete drawings you no longer need.
- Do heavy work only on `barstate.islast` when you only need the result on the newest bar.

Errors explained

When something is wrong, Zscript tells you the **line and column**, says what happened in plain English, and when it can, suggests a fix ("Did you mean ...?").

The three kinds

- **Errors** stop the script from running. The editor underlines the spot in red.
- **Warnings** let it run but point at something risky. Underlined in amber.
- **Notices** are short one-time notes on the chart, such as "Order book data needs Pro".

Common errors

Messages below are written the way the engine words them. Your names and numbers fill the gaps. Every code is listed in Error and warning codes.

E303 Unknown name

```
3:11  E303  Unknown name `smaa`.
            Did you mean `ta.sma`?
```

A typo. Names are case sensitive, and every built-in sits inside a namespace (`ta.` , `math.` , `str.`).

E309 Wrong type for an argument

```
5:8   E309  Argument `title` of `plot` must be string, got float.
```

You passed a number where text goes. Check the order of the arguments, or name them: `plot(x, title="Mine")`.

E201 Missing header

```
1:1   E201  Missing header.
            Put `//@zscript 1` on the first line.
```

Add the header line. Then make sure there is exactly one `indicator(...)` (E301 means it is missing, E302 means there are two).

E105 and E205 Indentation

```
6:1   E105  Indentation does not match any outer block.
```

Indent only under `if`, `for`, `while`, `switch` and function bodies, and line up the lines of one block exactly.

E305 Already declared

```
7:5   E305  `greens` is already declared in this scope.
            Use `:=` to change it.
```

A single `=` makes a new value; `:=` changes one that exists. E306 is the reverse: using `:=` before the value was ever made.

E209 One equals sign in a condition

```
4:8   E209  `=` used in a condition.
            Use `==` to compare, `:=` to reassign.
```

E340 Plot inside a block

```
9:5   E340  `plot` must be called at top level (not inside if/for/
functions).
```

Plots and their friends sit at the top of the script. To plot only sometimes, make the value `na` when you do not want it: `plot(cond ? x : na)`.

W101 A `ta` function that skips bars

```
12:9  W101  `ta.ema` is called only on some bars (inside `if`), so
its state skips the other bars.
            Compute it at top level and use the result inside the
`if`.
```

A warning, not an error. The script runs, but the numbers can be wrong. Do what the hint says.

Runtime errors (E4xx)

```
12:5  E404  Loop ran more than 100,000 times on one bar.
```

Shorten the loop, or use a built-in. See Limits. `E402` on a history look-back means you looked further back than `max_bars_back` allows; raise it in `indicator()`.

No value (na)

An `na` result is not an error. It means "nothing to show on this bar". See Bars and series. Division by zero gives `na`, not an error.

If you are stuck

Cut the script down to the smallest piece that still shows the problem. Add lines back one at a time until the error returns. The line it returns on is the cause.

Error and warning codes

Each message has a code, a line and column, and often a hint. Words in {braces} are filled in with your own names and numbers.

Lexer / header

Code	Meaning
E101	Unexpected character `{c}`.
E102	Unterminated string (closing quote missing).
E103	Bad color literal `{text}` (use #RRGGBB or #RRGGBBAA).
E104	Bad number `{text}`.
E105	Indentation does not match any outer block.
E106	Source is too large (limit 256 KB).
E201	Missing header. Hint: put `//@zscript 1` on the first line.
E202	Unsupported version `{n}`. Hint: use `//@zscript 1`.
E203	Unexpected `{token}`, expected {what}.
E204	Missing closing `)` / `]`.
E205	Expected an indented block after `{kw}`.
E206	`=>` default case must be last in switch.
E207	Nesting too deep (limit 128).
E208	`break` / `continue` outside a loop.
E209	`=` used in a condition. Hint: use `==` to compare, `:=` to reassign.
E210	Reserved word `{w}` cannot be used here.

Checker

Code	Meaning
E301	Missing `indicator(...)` declaration. Hint: add `indicator("My script")` after the header.
E302	`indicator()` declared twice.
E303	Unknown name `{x}`. Hint: Did you mean `{y}`?
E304	Unknown function `{f}`. Hint: Did you mean `{g}`?
E305	`{x}` is already declared in this scope. Hint: use `:=` to change it.
E306	Cannot reassign `{x}` with `:=` before it is declared. Hint: declare it with `=` first.
E307	`{f}` expects {n} arguments, got {m}.
E308	`{f}` has no parameter named `{p}`. Hint: Did you mean `{q}`?
E309	Argument `{p}` of `{f}` must be {type}, got {type2}.
E310	Cannot assign {type} to `{x}` declared as {type2}.
E311	`{x}` is {type}; cannot assign {type2}.
E312	Operator `{op}` cannot be used with {type} and {type2}.
E313	Condition must be bool or number, got {type}.
E314	Tuple has {n} values but {m} names.
E315	`{x}` is not a function.
E316	History `[]` needs a number of bars, got {type}.
E317	Built-in `{x}` cannot be reassigned.
E318	Function `{f}` calls itself (recursion is not allowed).
E319	Functions must be defined at top level.
E320	Method `{m}` does not exist for {type}. Hint: Did you mean `{n}`?
E321	Positional argument after a named one.

Code	Meaning
E322	Function `{f}` declared twice.
E330	Inputs must be called at top level with a literal default.
E340	`{f}` must be called at top level (not inside if/for/functions).
E341	Too many plots (limit 64).
E350	request.security expression uses the script variable `{x}` . Hint: compute it inside the expression, or move it into a function.
E351	lookahead is not allowed.
E352	Timeframe `{tf}` is lower than the chart timeframe.
E353	Too many request.security calls (limit 16).
W101	`{f}` is called only on some bars (inside `if`), so its state skips the other bars. Hint: compute it at top level and use the result inside the `if` .
W102	`{f}` inside a loop shares one state for every iteration.
W103	Variable `{x}` is never used.

Runtime

Code	Meaning
E401	Script took longer than 20 s.
E402	History `[{n}]` is beyond max_bars_back ({m}). Hint: raise max_bars_back in indicator().
E403	Too many operations on one bar (limit 1,000,000). Hint: a loop may be too large.
E404	Loop ran more than 100,000 times on one bar.
E405	Array too large (limit 100,000 items) / too many items in all arrays (1,000,000).
E406	Map too large (limit 50,000 keys).
E407	String too long (limit 4,096).
E408	Too many {kind} drawings is not an error: oldest is removed. (Tables over 20 -> E408.)
E409	Division by zero is NOT an error: result is na.
E410	Index {i} out of range for array of size {n}.
E411	Called a method on na (`{what}`).
E412	`{f}` length must be a positive number, got {v}.
E413	Too many bars (limit 50,000).
E414	Function calls nested too deep (limit 64).
E415	request.security data for {sym} {tf} is not available (warning-level notice, value na).
E499	Internal engine error (report this script to support).

Patterns every scripter uses

Once you know the pieces, a few patterns turn up again and again. Learn these and most scripts write themselves.

1. Compute first, decide after

Work out every indicator at the top of the script, on every bar. Then use the finished numbers inside `if` and `switch`. This keeps each `ta` function's memory complete.

```
//@zscript 1
indicator("Compute first", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
trendUp = fast > slow

plot(trendUp ? fast : slow, "Active line", color=trendUp ?
color.green : color.red)
```

2. Use `na` to mean "draw nothing"

A plot cannot sit inside an `if`. Instead make the value `na` on the bars where you want nothing.

```
//@zscript 1
indicator("Only when quiet", overlay=false)
r = (high - low) / close * 100
quiet = r < ta.sma(r, 20) * 0.7
plot(quiet ? r : na, "Quiet bars", style=plot.style_columns,
color=color.gray)
```

3. A flag that flips

Remember the last state with `var` and change it only when something happens.

```
//@zscript 1
indicator("Last cross direction", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
var int dir = 0
if ta.crossover(fast, slow)
    dir := 1
if ta.crossunder(fast, slow)
    dir := -1
bgcolor(dir == 1 ? color.new(color.green, 94) : dir == -1 ?
color.new(color.red, 94) : na)
```

4. Remember a price at the signal

`ta.valuewhen` hands you the value a series had when a condition was last true.

```
//@zscript 1
indicator("Price at last cross", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
lastCrossPrice = ta.valuewhen(ta.cross(fast, slow), close, 0)
plot(lastCrossPrice, "Close at last cross", color=color.yellow,
style=plot.style_stepline)
```

5. Normalize so scripts work on any symbol

Divide by price or by ATR so your numbers mean the same on every market.

```
//@zscript 1
indicator("Move in ATRs")
move = (close - close[10]) / ta.atr(14)
hline(0)
plot(move, "10-bar move in ATRs", style=plot.style_histogram,
color=move >= 0 ? color.green : color.red)
```

6. Smooth a noisy value

```
//@zscript 1
indicator("Smoothed volume")
plot(volume, "Volume", style=plot.style_columns,
color=color.new(color.gray, 60))
plot(ta.ema(volume, 10), "Smoothed", color=color.orange)
```

7. Limit how many drawings you make

Create a drawing only when it is new, and delete old ones.

```
//@zscript 1
indicator("One line for the latest high", overlay=true)
var line hiLine = na
ph = ta.pivothigh(high, 5, 5)
if not na(ph)
    line.delete(hiLine)
    hiLine := line.new(bar_index - 5, ph, bar_index, ph,
extend=extend.right, color=color.red)
```

8. Draw the summary only on the last bar

```
//@zscript 1
indicator("Last bar summary", overlay=true)
upBars = ta.sma(close > open ? 1.0 : 0.0, 50) * 100
if barstate.islast
    label.new(bar_index, high, "Up bars: " + str.tostring(upBars,
"#") + "%", style=label.style_label_down)
```

9. Reuse with a function

If you write the same three lines twice, make a function.

```
//@zscript 1
indicator("Two stochastics")
stoch1(len) => ta.stoch(close, high, low, len)
plot(stoch1(14), "Slow", color=color.blue)
plot(stoch1(5), "Fast", color=color.orange)
```

10. Guard data calls

Every call into `book`, `flow`, `gex` and `opt` can return `na`. Use `nz` when a missing value should count as zero, and `na()` when it should mean "skip".

```
//@zscript 1
indicator("Guarded delta")
d = flow.delta
plot(na(d) ? na : d, "Delta", style=plot.style_columns,
color=nz(d) >= 0 ? color.green : color.red)
```

Build four indicators step by step

The best way to learn is to build. Here are four indicators, from small to bigger. Type each step in and watch it grow.

Project 1. A trend ribbon

Goal: a line that is green in an up-trend and red in a down-trend.

Step 1. Draw the average.

```
//@zscript 1
indicator("Ribbon, step 1", overlay=true)
mid = ta.ema(close, 21)
plot(mid)
```

Step 2. Decide the trend: price above the average means up.

```
//@zscript 1
indicator("Ribbon, step 2", overlay=true)
mid = ta.ema(close, 21)
up = close > mid
plot(mid, color=up ? color.green : color.red, linewidth=2)
```

Step 3. Make the length a setting and add a soft band using ATR.

```
//@zscript 1
indicator("Trend ribbon", overlay=true)
len = input.int(21, "Length", minval=2)
width = input.float(1.0, "Band width in ATRs", minval=0.1,
step=0.1)
mid = ta.ema(close, len)
rng = ta.atr(14) * width
up = close > mid
tone = up ? color.green : color.red
p1 = plot(mid + rng, "Upper", color=color.new(tone, 80))
p2 = plot(mid - rng, "Lower", color=color.new(tone, 80))
plot(mid, "Middle", color=tone, linewidth=2)
fill(p1, p2, color.new(tone, 92))
```

You just used inputs, a condition, a color choice and a fill. That is most of the language.

Project 2. Previous day levels

Goal: draw yesterday's high, low and close as flat lines on a small chart.

Step 1. Ask for the daily values. `[1]` means the last finished day.

```
//@zscript 1
indicator("Prior day, step 1", overlay=true)
pdh = request.security(syminfo.tickerid, "D", high[1])
plot(pdh, "Prior day high", color=color.green,
style=plot.style_stepline)
```

Step 2. Add the low and the close, and label the newest bar.

```
//@zscript 1
indicator("Prior day levels", overlay=true)
pdh = request.security(syminfo.tickerid, "D", high[1])
pdl = request.security(syminfo.tickerid, "D", low[1])
pdc = request.security(syminfo.tickerid, "D", close[1])
plot(pdh, "Prior day high", color=color.green,
style=plot.style_stepline)
plot(pdl, "Prior day low", color=color.red,
style=plot.style_stepline)
plot(pdc, "Prior day close", color=color.gray,
style=plot.style_stepline)

if barstate.islast
    label.new(bar_index, pdh, "PDH", style=label.style_label_left,
color=color.green, textcolor=color.white)
    label.new(bar_index, pdl, "PDL", style=label.style_label_left,
color=color.red, textcolor=color.white)
```

Project 3. A volume pressure panel

Goal: a pane that shows how much of each bar's volume was at the ask, with a smoothed line. Needs order flow data, which follows your plan.

Step 1. The share of buying volume, per bar.

```
//@zscript 1
indicator("Buy share, step 1")
share = flow.buyvol / (flow.buyvol + flow.sellvol) * 100
hline(50)
plot(share, "Buy share %", style=plot.style_columns,
color=color.gray)
```

Step 2. Color by side and add a smooth line.

```
//@zscript 1
indicator("Buy share")
share = flow.buyvol / (flow.buyvol + flow.sellvol) * 100
hline(50)
plot(share, "Buy share %", style=plot.style_columns, color=share
>= 50 ? color.new(color.green, 40) : color.new(color.red, 40))
plot(ta.ema(share, 10), "Smoothed", color=color.white, linewidth=2)
```

When the plan lacks order flow data, `share` is `na`, nothing draws, and you see a short notice. The script itself is fine.

Project 4. A level board with alerts

Goal: show GEX levels as lines, a small table with the distance to each, and an alert when price crosses the flip level.

```
//@zscript 1
indicator("Level board", overlay=true)

plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
plot(gex.flip, "Flip", color=color.yellow,
style=plot.style_stepline)

var t = table.new(position.top_right, 2, 3)
if barstate.islast
    table.cell(t, 0, 0, "To call wall")
    table.cell(t, 1, 0, str.tostring(gex.callwall - close, "#.##"))
    table.cell(t, 0, 1, "To put wall")
    table.cell(t, 1, 1, str.tostring(close - gex.putwall, "#.##"))
    table.cell(t, 0, 2, "To flip")
    table.cell(t, 1, 2, str.tostring(close - gex.flip, "#.##"))

alertcondition(ta.crossover(close, gex.flip), "Crossed above
flip", "Price crossed above the flip level")
alertcondition(ta.crossunder(close, gex.flip), "Crossed below
flip", "Price crossed below the flip level")
```

GEX levels are updated a few minutes at a time, so this board shows the latest update, not an instant feed. Education only. Not financial advice.

Where to go from here

Open the Cookbook and change a recipe. Then try your own idea. When it works, think about publishing it.

Glossary

Alert. A rule that tells you when it becomes true.

Array. A list of values you fill and read yourself.

ATR. Average true range: the typical size of a bar.

Bar. One candle: its open, high, low, close and volume.

Bar index. The number of a bar, counting from 0 at the oldest.

Bool. A value that is `true` or `false`.

Built-in. A function or value that comes with Zscript, such as `ta.sma` or `close`.

CVD. Cumulative volume delta: the running total of delta.

Declaration. The `indicator(...)` line that names the script.

Delta. Volume traded at the ask minus volume traded at the bid.

Drawing. A label, line, box or table that you create and can change or delete.

Footprint. Volume traded at each price level inside a bar.

Function. A named piece of logic, such as `ta.sma` or one you write yourself.

GEX. Gamma exposure: a way of summarising option positions into price levels.

Header. The first line, `//@zscript 1`.

History. Older values of a series, read with `[n]`.

Input. A setting the person using the script can change.

Level two (L2). The order book: resting buy and sell orders at each price.

na. "Not available": no value on this bar.

Namespace. A group of built-ins sharing a prefix, such as `ta.` or `book.`.

Overlay. Drawing on the price chart instead of in a separate pane.

Pane. A chart area. Price is one pane; an indicator with `overlay=false` gets its own.

Pivot. A swing high or low, confirmed by bars on both sides.

Plot. A line, histogram or other shape with one value per bar.

Plan. Your TradeZoro plan, which decides which data kinds your scripts can read.

Protected script. A published script that people can use but not read.

Series. A value that has one number per bar and keeps its history.

Tuple. Several values returned together, taken apart with `[a, b] = f()`.

var. A keyword that makes a value remember itself from bar to bar.

ZoroOptions. TradeZoro's options summary: gamma nodes, ranges, IV rank and more.

Part 2. Reference

General

Core helpers: testing for missing values and changing types.

`bool(x)`

Converts to bool (na and 0 are false).

Returns: true or false

Parameter	Type
`x`	any

`color(x)`

Casts to color.

Returns: a color

Parameter	Type
`x`	any

`fixnan(x)`

The last non-na value of x. Call on every bar: it keeps its own running memory.

Returns: the same type as the first argument

Parameter	Type
`x`	any

`float(x)`

Converts to a float.

Returns: a number

Parameter	Type
`x`	any

`int(x)`

Converts to a whole number (truncates toward zero).

Returns: a whole number

Parameter	Type
`x`	any

`na(x)`

True if x is na (missing).

Returns: true or false

Parameter	Type
`x`	any

`nz(x, replacement=0)`

x, or replacement when x is na.

Returns: the same type as the first argument

Parameter	Type	Notes
`x`	any	
`replacement`	any	Default 0

`string(x)`

Converts to text.

Returns: text

Parameter	Type
`x`	any

`timestamp(year, month, day, hour=0, minute=0, second=0)`

Time in ms (UTC) for a date.

Returns: a whole number

Parameter	Type	Notes
`year`	int	
`month`	int	
`day`	int	
`hour`	int	Default 0
`minute`	int	Default 0
`second`	int	Default 0

Declaration and inputs

The indicator() line and every input.* setting.

`indicator(title, shorttitle="", overlay=false, format="price", precision=na, max_bars_back=500, max_labels_count=50, max_lines_count=50, max_boxes_count=50, max_tables_count=10, scale="right", timeframe="", explicit_plot_zorder=false)`

Declares the script: its title and where it draws. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`title`	string	
`shorttitle`	string	Default ""
`overlay`	bool	Default false
`format`	string	Default "price"
`precision`	int	Default na
`max_bars_back`	int	Default 500
`max_labels_count`	int	Default 50
`max_lines_count`	int	Default 50
`max_boxes_count`	int	Default 50
`max_tables_count`	int	Default 10
`scale`	string	Default "right"
`timeframe`	string	Default ""
`explicit_plot_zorder`	bool	Default false

`input(defval, title="", tooltip="", group="", inline="", confirm=false, display="all")`

A setting; its type comes from the default value. Call at the top level of the script.

Returns: the same type as the first argument

Parameter	Type	Notes
`defval`	any	
`title`	string	Default ""
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.bool(defval, title="", tooltip="", group="", inline="", confirm=false, display="all")`

An on/off setting. Call at the top level of the script.

Returns: true or false

Parameter	Type	Notes
`defval`	bool	
`title`	string	Default ""
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.color(defval, title="", tooltip="", group="", inline="", confirm=false, display="all")`

A color setting. Call at the top level of the script.

Returns: a color

Parameter	Type	Notes
`defval`	color	
`title`	string	Default ""
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.float(defval, title="", minval=na, maxval=na, step=na, options=na, tooltip="", group="", inline="", confirm=false, display="all")`

A decimal-number setting. Call at the top level of the script.

Returns: a number

Parameter	Type	Notes
`defval`	float	
`title`	string	Default ""
`minval`	float	Default na
`maxval`	float	Default na
`step`	float	Default na
`options`	array	Default na
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.int(defval, title="", minval=na, maxval=na, step=1, options=na, tooltip="", group="", inline="", confirm=false, display="all")`

A whole-number setting. Call at the top level of the script.

Returns: a whole number

Parameter	Type	Notes
`defval`	int	
`title`	string	Default ""
`minval`	int	Default na
`maxval`	int	Default na
`step`	int	Default 1
`options`	array	Default na
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.price(defval, title="", tooltip="", group="", inline="", confirm=false, display="all")`

A price-level setting. Call at the top level of the script.

Returns: a number

Parameter	Type	Notes
`defval`	float	
`title`	string	Default ""
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.source(defval=close, title="", tooltip="", group="", inline="", confirm=false, display="all")`

A price source setting (open, high, low, close, volume, hl2, hlc3, ohlc4, hlcc4). Call at the top level of the script.

Returns: a number

Parameter	Type	Notes
`defval`	float	Default close
`title`	string	Default ""
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.string(defval, title="", options=na, tooltip="", group="", inline="", confirm=false, display="all")`

A text setting (or a dropdown with options). Call at the top level of the script.

Returns: text

Parameter	Type	Notes
`defval`	string	
`title`	string	Default ""
`options`	array	Default na
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.symbol(defval="", title="", tooltip="", group="", inline="", confirm=false, display="all")`

A symbol setting ("" = chart symbol). Call at the top level of the script.

Returns: text

Parameter	Type	Notes
`defval`	string	Default ""
`title`	string	Default ""
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.text_area(defval, title="", tooltip="", group="", confirm=false, display="all")`

A multi-line text setting. Call at the top level of the script.

Returns: text

Parameter	Type	Notes
`defval`	string	
`title`	string	Default ""
`tooltip`	string	Default ""
`group`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

`input.timeframe(defval="", title="", options=na, tooltip="", group="", inline="", confirm=false, display="all")`

A timeframe setting ("" = chart timeframe). Call at the top level of the script.

Returns: text

Parameter	Type	Notes
`defval`	string	Default ""
`title`	string	Default ""
`options`	array	Default na
`tooltip`	string	Default ""
`group`	string	Default ""
`inline`	string	Default ""
`confirm`	bool	Default false
`display`	string	Default "all"

Indicators (ta)

Moving averages, oscillators, volatility and pattern helpers. Each call keeps its own running memory, so call them on every bar.

`ta.accdist`

Accumulation/distribution.

Returns: a number

`ta.alma(source, length, offset=0.85, sigma=6)`

Arnaud Legoux moving average. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	
`length`	int	
`offset`	float	Default 0.85
`sigma`	float	Default 6

`ta.atr(length)`

Average true range (RMA of the true range). Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`length`	int

`ta.barssince(condition)`

Bars since condition was last true (na if never). Call on every bar: it keeps its own running memory.

Returns: a whole number

Parameter	Type
`condition`	bool

`ta.bb(source, length, mult)`

Bollinger Bands: middle, upper, lower. Call on every bar: it keeps its own running memory.

Returns: several values: [float,float,float]

Parameter	Type
`source`	num
`length`	int
`mult`	float

`ta.bbw(source, length, mult)`

Bollinger Band width as a percent of the middle band. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int
`mult`	float

`ta.cci(source, length)`

Commodity channel index. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.change(source, length=1)`

source minus its value length bars ago (for bool: true if it changed). Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	any	
`length`	int	Default 1

`ta.cmo(source, length)`

Chande momentum oscillator. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.correlation(source1, source2, length)`

Correlation coefficient, -1 to 1. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source1`	num
`source2`	num
`length`	int

`ta.cross(a, b)`

True on the bar where a and b cross either way. Call on every bar: it keeps its own running memory.

Returns: true or false

Parameter	Type
`a`	num
`b`	num

`ta.crossover(a, b)`

True on the bar where a crosses above b. Call on every bar: it keeps its own running memory.

Returns: true or false

Parameter	Type
`a`	num
`b`	num

`ta.crossunder(a, b)`

True on the bar where a crosses below b. Call on every bar: it keeps its own running memory.

Returns: true or false

Parameter	Type
`a`	num
`b`	num

`ta.cum(source)`

Running total of source since the first bar. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num

`ta.dev(source, length)`

Mean absolute deviation from the SMA. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.dmi(diLength, adxSmoothing)`

Directional movement: +DI, -DI and ADX. Call on every bar: it keeps its own running memory.

Returns: several values: [float,float,float]

Parameter	Type
`diLength`	int
`adxSmoothing`	int

`ta.ema(source, length)`

Exponential moving average (alpha 2/(length+1), starts from an SMA). Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.falling(source, length)`

True if source fell on each of the last length bars. Call on every bar: it keeps its own running memory.

Returns: true or false

Parameter	Type
`source`	num
`length`	int

`ta.highest(source=high, length)`

Highest value of the last length bars. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	Default high
`length`	int	

`ta.highestbars(source=high, length)`

Offset (0 or negative) to the highest bar of the last length bars. Call on every bar: it keeps its own running memory.

Returns: a whole number

Parameter	Type	Notes
`source`	num	Default high
`length`	int	

`ta.hma(source, length)`

Hull moving average. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.kc(source, length, mult, useTrueRange=true)`

Keltner Channels: middle (EMA), upper, lower. Call on every bar: it keeps its own running memory.

Returns: several values: [float,float,float]

Parameter	Type	Notes
`source`	num	
`length`	int	
`mult`	float	
`useTrueRange`	bool	Default true

`ta.kcw(source, length, mult, useTrueRange=true)`

Keltner Channel width. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	
`length`	int	
`mult`	float	
`useTrueRange`	bool	Default true

`ta.linreg(source, length, offset=0)`

Linear regression value (least squares line) at offset bars back. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	
`length`	int	
`offset`	int	Default 0

`ta.lowest(source=low, length)`

Lowest value of the last length bars. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	Default low
`length`	int	

`ta.lowestbars(source=low, length)`

Offset (0 or negative) to the lowest bar of the last length bars. Call on every bar: it keeps its own running memory.

Returns: a whole number

Parameter	Type	Notes
`source`	num	Default low
`length`	int	

`ta.macd(source, fastlen=12, slowlen=26, siglen=9)`

MACD line, signal line and histogram. Call on every bar: it keeps its own running memory.

Returns: several values: [float,float,float]

Parameter	Type	Notes
`source`	num	
`fastlen`	int	Default 12
`slowlen`	int	Default 26
`siglen`	int	Default 9

`ta.median(source, length)`

Median of the last length values. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.mfi(source, length)`

Money flow index, 0 to 100. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.mode(source, length)`

Most frequent value of the last length values. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.mom(source, length)`

Momentum: source minus source length bars ago. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.obv`

On-balance volume.

Returns: a number

`ta.percentrank(source, length)`

Percent of the previous length values that are at or below the current one. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.pivothigh(source=high, leftbars, rightbars)`

Pivot high price, known rightbars bars after the pivot (na otherwise). Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	Default high
`leftbars`	int	
`rightbars`	int	

`ta.pivotlow(source=low, leftbars, rightbars)`

Pivot low price, known rightbars bars after the pivot (na otherwise). Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	Default low
`leftbars`	int	
`rightbars`	int	

`ta.range(source, length)`

Highest minus lowest of the last length values. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.rising(source, length)`

True if source rose on each of the last length bars. Call on every bar: it keeps its own running memory.

Returns: true or false

Parameter	Type
`source`	num
`length`	int

`ta.rma(source, length)`

Wilder's moving average (alpha 1/length, starts from an SMA). Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.roc(source, length)`

Rate of change in percent. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.rsi(source, length)`

Relative strength index, 0 to 100. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.sar(start, inc, max)`

Parabolic SAR. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`start`	float
`inc`	float
`max`	float

`ta.sma(source, length)`

Simple moving average of the last length values. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.stdev(source, length, biased=true)`

Standard deviation. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	
`length`	int	
`biased`	bool	Default true

`ta.stoch(source, high, low, length)`

Stochastic: where source sits in the high-low range of length bars, 0 to 100. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`high`	num
`low`	num
`length`	int

`ta.supertrend(factor, atrPeriod)`

SuperTrend line and direction (-1 up trend, 1 down trend). Call on every bar: it keeps its own running memory.

Returns: several values: [float,int]

Parameter	Type
`factor`	float
`atrPeriod`	int

`ta.swma(source)`

Symmetric 4-bar weighted moving average. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num

`ta.tr(handle_na=false)`

True range of this bar. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`handle_na`	bool	Default false

`ta.tr`

True range of this bar (na on the first bar).

Returns: a number

`ta.tsi(source, short_length, long_length)`

True strength index, -1 to 1. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`short_length`	int
`long_length`	int

`ta.valuewhen(condition, source, occurrence=0)`

Value of source on the bar where condition was true (occurrence 0 = most recent). Call on every bar: it keeps its own running memory.

Returns: arg1

Parameter	Type	Notes
`condition`	bool	
`source`	any	
`occurrence`	int	Default 0

`ta.variance(source, length, biased=true)`

Variance. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	
`length`	int	
`biased`	bool	Default true

`ta.vwap(source=hlc3, anchor="D")`

Volume-weighted average price, restarting each day ("D"), week ("W") or month ("M"). Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`source`	num	Default hlc3
`anchor`	string	Default "D"

`ta.vwap`

Daily volume-weighted average price of hlc3.

Returns: a number

`ta.vwma(source, length)`

Volume-weighted moving average. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.wma(source, length)`

Linearly weighted moving average. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`ta.wpr(length)`

Williams %R, -100 to 0. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`length`	int

Math

Numbers and arithmetic.

`math.abs(x)`

Absolute value.

Returns: a number

Parameter	Type
`x`	num

`math.acos(x)`

Arc cosine.

Returns: a number

Parameter	Type
`x`	num

`math.asin(x)`

Arc sine.

Returns: a number

Parameter	Type
`x`	num

`math.atan(x)`

Arc tangent.

Returns: a number

Parameter	Type
`x`	num

`math.avg(...values)`

Average of the values.

Returns: a number

Parameter	Type
`values`	num

`math.ceil(x)`

Rounds up.

Returns: a whole number

Parameter	Type
`x`	num

`math.cos(x)`

Cosine (radians).

Returns: a number

Parameter	Type
`x`	num

`math.exp(x)`

e to the power x.

Returns: a number

Parameter	Type
`x`	num

`math.floor(x)`

Rounds down.

Returns: a whole number

Parameter	Type
`x`	num

`math.log(x)`

Natural logarithm.

Returns: a number

Parameter	Type
`x`	num

`math.log10(x)`

Base-10 logarithm.

Returns: a number

Parameter	Type
`x`	num

`math.max(...values)`

Largest of the values.

Returns: a number

Parameter	Type
`values`	num

`math.min(...values)`

Smallest of the values.

Returns: a number

Parameter	Type
`values`	num

`math.pow(base, exponent)`

base to the power exponent.

Returns: a number

Parameter	Type
`base`	num
`exponent`	num

`math.random(min=0, max=1, seed=na)`

A repeatable pseudo-random number between min and max. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type	Notes
`min`	num	Default 0
`max`	num	Default 1
`seed`	int	Default na

`math.round(x, precision=na)`

Rounds to the nearest whole number, or to precision decimals.

Returns: a number

Parameter	Type	Notes
`x`	num	
`precision`	int	Default na

`math.round_to_mintick(x)`

Rounds to the symbol's tick size.

Returns: a number

Parameter	Type
`x`	num

`math.sign(x)`

-1, 0 or 1.

Returns: a number

Parameter	Type
`x`	num

`math.sin(x)`

Sine (radians).

Returns: a number

Parameter	Type
`x`	num

`math.sqrt(x)`

Square root.

Returns: a number

Parameter	Type
`x`	num

`math.sum(source, length)`

Sum of the last length values. Call on every bar: it keeps its own running memory.

Returns: a number

Parameter	Type
`source`	num
`length`	int

`math.tan(x)`

Tangent (radians).

Returns: a number

Parameter	Type
`x`	num

`math.todegrees(x)`

Radians to degrees.

Returns: a number

Parameter	Type
`x`	num

`math.toradians(x)`

Degrees to radians.

Returns: a number

Parameter	Type
`x`	num

Text (str)

Building and reading text.

`str.contains(s, sub)`

True if s contains sub.

Returns: true or false

Parameter	Type
`s`	string
`sub`	string

`str.endswith(s, suffix)`

True if s ends with suffix.

Returns: true or false

Parameter	Type
`s`	string
`suffix`	string

`str.format(formatString, ...args)`

Fills {0}, {1}, ... placeholders.

Returns: text

Parameter	Type
`formatString`	string
`args`	any

`str.length(s)`

Number of characters.

Returns: a whole number

Parameter	Type
`s`	string

`str.lower(s)`

Lower case.

Returns: text

Parameter	Type
`s`	string

`str.pos(s, sub)`

Position of sub in s (na if absent).

Returns: a whole number

Parameter	Type
`s`	string
`sub`	string

`str.repeat(s, repeat, separator="")`

s repeated n times.

Returns: text

Parameter	Type	Notes
`s`	string	
`repeat`	int	
`separator`	string	Default ""

`str.replace(s, target, replacement, occurrence=0)`

Replaces one occurrence of target.

Returns: text

Parameter	Type	Notes
`s`	string	
`target`	string	
`replacement`	string	
`occurrence`	int	Default 0

`str.replace_all(s, target, replacement)`

Replaces every occurrence of target.

Returns: text

Parameter	Type
`s`	string
`target`	string
`replacement`	string

`str.split(s, separator)`

Splits text into an array.

Returns: an array of string

Parameter	Type
`s`	string
`separator`	string

`str.startswith(s, prefix)`

True if s starts with prefix.

Returns: true or false

Parameter	Type
`s`	string
`prefix`	string

`str.substring(s, begin, end=na)`

Part of s from begin up to (not including) end.

Returns: text

Parameter	Type	Notes
`s`	string	
`begin`	int	
`end`	int	Default na

`str.tonumber(s)`

Parses text as a number (na if it is not one).

Returns: a number

Parameter	Type
`s`	string

`str.tostring(value, format="")`

Formats a value as text ("#.##", "0.00", "percent", "price", "volume").

Returns: text

Parameter	Type	Notes
`value`	any	
`format`	string	Default ""

`str.trim(s)`

Removes spaces at both ends.

Returns: text

Parameter	Type
`s`	string

`str.upper(s)`

Upper case.

Returns: text

Parameter	Type
`s`	string

Colors

Named colors and ways to make your own.

`color.b(c)`

Blue part, 0-255.

Returns: a number

Parameter	Type
`c`	color

`color.from_gradient(value, bottom_value, top_value, bottom_color, top_color)`

A color between two colors, by where value sits between two values.

Returns: a color

Parameter	Type
`value`	num
`bottom_value`	num
`top_value`	num
`bottom_color`	color
`top_color`	color

`color.g(c)`

Green part, 0-255.

Returns: a number

Parameter	Type
`c`	color

`color.new(c, transp=0)`

The same color with transparency 0 (solid) to 100 (invisible).

Returns: a color

Parameter	Type	Notes
`c`	color	
`transp`	num	Default 0

`color.r(c)`

Red part, 0-255.

Returns: a number

Parameter	Type
`c`	color

`color.rgb(red, green, blue, transp=0)`

A color from red, green, blue (0-255).

Returns: a color

Parameter	Type	Notes
`red`	num	
`green`	num	
`blue`	num	
`transp`	num	Default 0

`color.t(c)`

Transparency, 0-100.

Returns: a number

Parameter	Type
`c`	color

Arrays

Lists of values. Method style works too: prices.push(close).

`array.abs(id)`

A new array with absolute values.

Returns: the same type as the first argument

Parameter	Type
`id`	array

`array.avg(id)`

Average of the items.

Returns: a number

Parameter	Type
`id`	array

`array.clear(id)`

Removes every item.

Returns: nothing

Parameter	Type
`id`	array

`array.concat(id1, id2)`

Appends the second array to the first.

Returns: the same type as the first argument

Parameter	Type
`id1`	array
`id2`	array

`array.copy(id)`

A copy of the array.

Returns: the same type as the first argument

Parameter	Type
`id`	array

`array.fill(id, value, index_from=0, index_to=na)`

Sets items in a range to value.

Returns: nothing

Parameter	Type	Notes
`id`	array	
`value`	any	
`index_from`	int	Default 0
`index_to`	int	Default na

`array.first(id)`

First item.

Returns: elem0

Parameter	Type
`id`	array

`array.from(...values)`

A new array holding the given values.

Returns: fromarr

Parameter	Type
`values`	any

`array.get(id, index)`

The item at index (-1 = last).

Returns: elem0

Parameter	Type
`id`	array
`index`	int

`array.includes(id, value)`

True if the value is in the array.

Returns: true or false

Parameter	Type
`id`	array
`value`	any

`array.indexof(id, value)`

First index of value (-1 if absent).

Returns: a whole number

Parameter	Type
`id`	array
`value`	any

`array.insert(id, index, value)`

Inserts an item at index.

Returns: nothing

Parameter	Type
`id`	array
`index`	int
`value`	any

`array.join(id, separator="")`

The items joined as text.

Returns: text

Parameter	Type	Notes
`id`	array	
`separator`	string	Default ""

`array.last(id)`

Last item.

Returns: elem0

Parameter	Type
`id`	array

`array.lastindexof(id, value)`

Last index of value (-1 if absent).

Returns: a whole number

Parameter	Type
`id`	array
`value`	any

`array.max(id)`

Largest item.

Returns: a number

Parameter	Type
`id`	array

`array.median(id)`

Median of the items.

Returns: a number

Parameter	Type
`id`	array

`array.min(id)`

Smallest item.

Returns: a number

Parameter	Type
`id`	array

`array.new(size=0, initial_value=na)`

A new array (write the type: array.new<float>(...)).

Returns: newarr

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	any	Default na

`array.new_bool(size=0, initial_value=false)`

A new array of bools.

Returns: an array of bool

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	bool	Default false

`array.new_box(size=0, initial_value=na)`

A new array of boxes.

Returns: an array of box

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	box	Default na

`array.new_color(size=0, initial_value=na)`

A new array of colors.

Returns: an array of color

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	color	Default na

`array.new_float(size=0, initial_value=na)`

A new array of floats.

Returns: an array of float

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	float	Default na

`array.new_int(size=0, initial_value=na)`

A new array of ints.

Returns: an array of int

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	int	Default na

`array.new_label(size=0, initial_value=na)`

A new array of labels.

Returns: an array of label

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	label	Default na

`array.new_line(size=0, initial_value=na)`

A new array of lines.

Returns: an array of line

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	line	Default na

`array.new_string(size=0, initial_value=na)`

A new array of strings.

Returns: an array of string

Parameter	Type	Notes
`size`	int	Default 0
`initial_value`	string	Default na

`array.pop(id)`

Removes and returns the last item.

Returns: elem0

Parameter	Type
`id`	array

`array.push(id, value)`

Adds an item at the end.

Returns: nothing

Parameter	Type
`id`	array
`value`	any

`array.range(id)`

Largest minus smallest item.

Returns: a number

Parameter	Type
`id`	array

`array.remove(id, index)`

Removes and returns the item at index.

Returns: elem0

Parameter	Type
`id`	array
`index`	int

`array.reverse(id)`

Reverses the order in place.

Returns: nothing

Parameter	Type
`id`	array

`array.set(id, index, value)`

Replaces the item at index.

Returns: nothing

Parameter	Type
`id`	array
`index`	int
`value`	any

`array.shift(id)`

Removes and returns the first item.

Returns: elem0

Parameter	Type
`id`	array

`array.size(id)`

Number of items.

Returns: a whole number

Parameter	Type
`id`	array

`array.slice(id, index_from, index_to=na)`

A copy of part of the array.

Returns: the same type as the first argument

Parameter	Type	Notes
`id`	array	
`index_from`	int	
`index_to`	int	Default na

`array.sort(id, order="ascending")`

Sorts in place (order.ascending or order.descending).

Returns: nothing

Parameter	Type	Notes
`id`	array	
`order`	string	Default "ascending"

`array.sort_indices(id, order="ascending")`

Indices that would sort the array.

Returns: an array of int

Parameter	Type	Notes
`id`	array	
`order`	string	Default "ascending"

`array.stdev(id, biased=true)`

Standard deviation of the items.

Returns: a number

Parameter	Type	Notes
`id`	array	
`biased`	bool	Default true

`array.sum(id)`

Sum of the items.

Returns: a number

Parameter	Type
`id`	array

`array.unshift(id, value)`

Adds an item at the start.

Returns: nothing

Parameter	Type
`id`	array
`value`	any

`array.variance(id, biased=true)`

Variance of the items.

Returns: a number

Parameter	Type	Notes
`id`	array	
`biased`	bool	Default true

Maps

Lookup tables from a key to a value.

`map.clear(id)`

Removes every key.

Returns: nothing

Parameter	Type
`id`	map

`map.contains(id, key)`

True if the key is present.

Returns: true or false

Parameter	Type
`id`	map
`key`	any

`map.copy(id)`

A copy of the map.

Returns: the same type as the first argument

Parameter	Type
`id`	map

`map.get(id, key)`

The value for key (na if absent).

Returns: vals0

Parameter	Type
`id`	map
`key`	any

`map.keys(id)`

An array of the keys, in insertion order.

Returns: keys0

Parameter	Type
`id`	map

`map.new()`

A new map (write the types: map.new<string, float>()).

Returns: newmap

`map.put(id, key, value)`

Sets the value for key.

Returns: nothing

Parameter	Type
`id`	map
`key`	any
`value`	any

`map.put_all(id, id2)`

Copies every key of id2 into id.

Returns: nothing

Parameter	Type
`id`	map
`id2`	map

`map.remove(id, key)`

Removes the key and returns its value.

Returns: vals0

Parameter	Type
`id`	map
`key`	any

`map.size(id)`

Number of keys.

Returns: a whole number

Parameter	Type
`id`	map

`map.values(id)`

An array of the values, in insertion order.

Returns: valarr0

Parameter	Type
`id`	map

Plots and chart colors

Everything that draws one value per bar. Call these at the top level of your script.

`barcolor(color, offset=0, editable=true, show_last=na, title="", display="all")`

Colors the candles. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`color`	color	
`offset`	int	Default 0
`editable`	bool	Default true
`show_last`	int	Default na
`title`	string	Default ""
`display`	string	Default "all"

`bgcolor(color, offset=0, editable=true, show_last=na, title="", display="all")`

Colors the chart background of each bar. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`color`	color	
`offset`	int	Default 0
`editable`	bool	Default true
`show_last`	int	Default na
`title`	string	Default ""
`display`	string	Default "all"

`fill(plot1, plot2, color=na, title="", editable=true, show_last=na, fillgaps=false, display="all")`

Fills the space between two plots or two hlines. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`plot1`	any	
`plot2`	any	
`color`	color	Default na
`title`	string	Default ""
`editable`	bool	Default true
`show_last`	int	Default na
`fillgaps`	bool	Default false
`display`	string	Default "all"

`hline(price, title="", color=#787b86, linestyle="dashed", linewidth=1, editable=true, display="all")`

Draws a horizontal line at a fixed price. Call at the top level of the script.

Returns: hlineid

Parameter	Type	Notes
`price`	num	
`title`	string	Default ""
`color`	color	Default #787b86
`linestyle`	string	Default "dashed"
`linewidth`	int	Default 1
`editable`	bool	Default true
`display`	string	Default "all"

`plot(series, title="", color=na, linewidth=1, style="line", trackprice=false, histbase=0, offset=0, join=false, editable=true, show_last=na, display="all", format="", precision=na)`

Draws a series as a line (or histogram, columns, circles, area...). Call at the top level of the script.

Returns: plotid

Parameter	Type	Notes
`series`	num	
`title`	string	Default ""
`color`	color	Default na
`linewidth`	int	Default 1
`style`	string	Default "line"
`trackprice`	bool	Default false
`histbase`	float	Default 0
`offset`	int	Default 0
`join`	bool	Default false
`editable`	bool	Default true
`show_last`	int	Default na
`display`	string	Default "all"
`format`	string	Default ""
`precision`	int	Default na

`plotarrow(series, title="", colorup=#089981, colordown=#f23645, offset=0, minheight=5, maxheight=100, editable=true, show_last=na, display="all")`

Draws up arrows for positive values and down arrows for negative values. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`series`	num	
`title`	string	Default ""
`colorup`	color	Default #089981
`colordown`	color	Default #f23645
`offset`	int	Default 0
`minheight`	int	Default 5
`maxheight`	int	Default 100
`editable`	bool	Default true
`show_last`	int	Default na
`display`	string	Default "all"

`plotbar(open, high, low, close, title="", color=na, editable=true, show_last=na, display="all")`

Draws OHLC bars. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`open`	num	
`high`	num	
`low`	num	
`close`	num	
`title`	string	Default ""
`color`	color	Default na
`editable`	bool	Default true
`show_last`	int	Default na
`display`	string	Default "all"

`plotcandle(open, high, low, close, title="", color=na, wickcolor=na, editable=true, show_last=na, bordercolor=na, display="all")`

Draws candles. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`open`	num	
`high`	num	
`low`	num	
`close`	num	
`title`	string	Default ""
`color`	color	Default na
`wickcolor`	color	Default na
`editable`	bool	Default true
`show_last`	int	Default na
`bordercolor`	color	Default na
`display`	string	Default "all"

`plotchar(series, title="", char="•", location="abovebar", color=na, offset=0, text="", textcolor=na, editable=true, size="auto", show_last=na, display="all")`

Draws a character on bars where series is true (or not na). Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`series`	any	
`title`	string	Default ""
`char`	string	Default "•"
`location`	string	Default "abovebar"
`color`	color	Default na
`offset`	int	Default 0
`text`	string	Default ""
`textcolor`	color	Default na
`editable`	bool	Default true
`size`	string	Default "auto"
`show_last`	int	Default na
`display`	string	Default "all"

`plotshape(series, title="", style="circle", location="abovebar", color=na, offset=0, text="", textcolor=na, editable=true, size="auto", show_last=na, display="all")`

Draws a shape on bars where series is true (or not na). Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`series`	any	
`title`	string	Default ""
`style`	string	Default "circle"
`location`	string	Default "abovebar"
`color`	color	Default na
`offset`	int	Default 0
`text`	string	Default ""
`textcolor`	color	Default na
`editable`	bool	Default true
`size`	string	Default "auto"
`show_last`	int	Default na
`display`	string	Default "all"

Labels

Text notes at a price and bar.

`label.copy(id)`

A copy of the label.

Returns: label

Parameter	Type
`id`	label

`label.delete(id)`

Removes the label.

Returns: nothing

Parameter	Type
`id`	label

`label.get_text(id)`

The label's text.

Returns: text

Parameter	Type
`id`	label

`label.get_x(id)`

The label's x.

Returns: a whole number

Parameter	Type
`id`	label

`label.get_y(id)`

The label's y.

Returns: a number

Parameter	Type
`id`	label

`label.new(x, y, text="", xloc="bar_index", yloc="price", color=na, style="label_down", textcolor=na, size="normal", textalign="center", tooltip="", text_font_family="default")`

Creates a label and returns its id.

Returns: label

Parameter	Type	Notes
`x`	num	
`y`	num	
`text`	string	Default ""
`xloc`	string	Default "bar_index"
`yloc`	string	Default "price"
`color`	color	Default na
`style`	string	Default "label_down"
`textcolor`	color	Default na
`size`	string	Default "normal"
`textalign`	string	Default "center"
`tooltip`	string	Default ""
`text_font_family`	string	Default "default"

`label.set_color(id, color)`

Changes the background color.

Returns: nothing

Parameter	Type
`id`	label
`color`	color

`label.set_size(id, size)`

Changes the size.

Returns: nothing

Parameter	Type
`id`	label
`size`	string

`label.set_style(id, style)`

Changes the style.

Returns: nothing

Parameter	Type
`id`	label
`style`	string

`label.set_text(id, text)`

Changes the text.

Returns: nothing

Parameter	Type
`id`	label
`text`	string

`label.set_textalign(id, textalign)`

Changes the text alignment.

Returns: nothing

Parameter	Type
`id`	label
`textalign`	string

`label.set_textcolor(id, textcolor)`

Changes the text color.

Returns: nothing

Parameter	Type
`id`	label
`textcolor`	color

`label.set_tooltip(id, tooltip)`

Changes the tooltip.

Returns: nothing

Parameter	Type
`id`	label
`tooltip`	string

`label.set_x(id, x)`

Moves the label horizontally.

Returns: nothing

Parameter	Type
`id`	label
`x`	num

`label.set_xloc(id, x, xloc)`

Changes x and how it is read.

Returns: nothing

Parameter	Type
`id`	label
`x`	num
`xloc`	string

`label.set_xy(id, x, y)`

Moves the label.

Returns: nothing

Parameter	Type
`id`	label
`x`	num
`y`	num

`label.set_y(id, y)`

Moves the label vertically.

Returns: nothing

Parameter	Type
`id`	label
`y`	num

`label.set_yloc(id, yloc)`

Changes how y is read.

Returns: nothing

Parameter	Type
`id`	label
`yloc`	string

Lines

Line segments between two points.

`line.copy(id)`

A copy of the line.

Returns: line

Parameter	Type
`id`	line

`line.delete(id)`

Removes the line.

Returns: nothing

Parameter	Type
`id`	line

`line.get_price(id, x)`

The line's price at bar x.

Returns: a number

Parameter	Type
`id`	line
`x`	num

`line.get_x1(id)`

First point x.

Returns: a whole number

Parameter	Type
`id`	line

`line.get_x2(id)`

Second point x.

Returns: a whole number

Parameter	Type
`id`	line

`line.get_y1(id)`

First point y.

Returns: a number

Parameter	Type
`id`	line

`line.get_y2(id)`

Second point y.

Returns: a number

Parameter	Type
`id`	line

`line.new(x1, y1, x2, y2, xloc="bar_index", extend="none", color=na, style="solid", width=1)`

Creates a line and returns its id.

Returns: line

Parameter	Type	Notes
`x1`	num	
`y1`	num	
`x2`	num	
`y2`	num	
`xloc`	string	Default "bar_index"
`extend`	string	Default "none"
`color`	color	Default na
`style`	string	Default "solid"
`width`	int	Default 1

`line.set_color(id, color)`

Changes the color.

Returns: nothing

Parameter	Type
`id`	line
`color`	color

`line.set_extend(id, extend)`

Extends the line left, right, both or none.

Returns: nothing

Parameter	Type
`id`	line
`extend`	string

`line.set_style(id, style)`

Changes the style.

Returns: nothing

Parameter	Type
`id`	line
`style`	string

`line.set_width(id, width)`

Changes the width.

Returns: nothing

Parameter	Type
`id`	line
`width`	int

`line.set_x1(id, x)`

Moves the first point horizontally.

Returns: nothing

Parameter	Type
`id`	line
`x`	num

`line.set_x2(id, x)`

Moves the second point horizontally.

Returns: nothing

Parameter	Type
`id`	line
`x`	num

`line.set_xloc(id, x1, x2, xloc)`

Changes x1, x2 and how they are read.

Returns: nothing

Parameter	Type
`id`	line
`x1`	num
`x2`	num
`xloc`	string

`line.set_xy1(id, x, y)`

Moves the first point.

Returns: nothing

Parameter	Type
`id`	line
`x`	num
`y`	num

`line.set_xy2(id, x, y)`

Moves the second point.

Returns: nothing

Parameter	Type
`id`	line
`x`	num
`y`	num

`line.set_y1(id, y)`

Moves the first point vertically.

Returns: nothing

Parameter	Type
`id`	line
`y`	num

`line.set_y2(id, y)`

Moves the second point vertically.

Returns: nothing

Parameter	Type
`id`	line
`y`	num

Boxes

Rectangles between two corners.

`box.copy(id)`

A copy of the box.

Returns: box

Parameter	Type
`id`	box

`box.delete(id)`

Removes the box.

Returns: nothing

Parameter	Type
`id`	box

`box.get_bottom(id)`

Bottom edge.

Returns: a number

Parameter	Type
`id`	box

`box.get_left(id)`

Left edge.

Returns: a whole number

Parameter	Type
`id`	box

`box.get_right(id)`

Right edge.

Returns: a whole number

Parameter	Type
`id`	box

`box.get_top(id)`

Top edge.

Returns: a number

Parameter	Type
`id`	box

`box.new(left, top, right, bottom, border_color=na, border_width=1, border_style="solid", extend="none", xloc="bar_index", bgcolor=na, text="", text_size="auto", text_color=na, text_halign="center", text_valign="center")`

Creates a box and returns its id.

Returns: box

Parameter	Type	Notes
`left`	num	
`top`	num	
`right`	num	
`bottom`	num	
`border_color`	color	Default na
`border_width`	int	Default 1
`border_style`	string	Default "solid"
`extend`	string	Default "none"
`xloc`	string	Default "bar_index"
`bgcolor`	color	Default na
`text`	string	Default ""
`text_size`	string	Default "auto"
`text_color`	color	Default na
`text_halign`	string	Default "center"
`text_valign`	string	Default "center"

`box.set_bgcolor(id, color)`

Changes the fill color.

Returns: nothing

Parameter	Type
`id`	box
`color`	color

`box.set_border_color(id, color)`

Changes the border color.

Returns: nothing

Parameter	Type
`id`	box
`color`	color

`box.set_border_style(id, style)`

Changes the border style.

Returns: nothing

Parameter	Type
`id`	box
`style`	string

`box.set_border_width(id, width)`

Changes the border width.

Returns: nothing

Parameter	Type
`id`	box
`width`	int

`box.set_bottom(id, bottom)`

Moves the bottom edge.

Returns: nothing

Parameter	Type
`id`	box
`bottom`	num

`box.set_extend(id, extend)`

Extends the box left, right, both or none.

Returns: nothing

Parameter	Type
`id`	box
`extend`	string

`box.set_left(id, left)`

Moves the left edge.

Returns: nothing

Parameter	Type
`id`	box
`left`	num

`box.set_lefttop(id, left, top)`

Moves the top-left corner.

Returns: nothing

Parameter	Type
`id`	box
`left`	num
`top`	num

`box.set_right(id, right)`

Moves the right edge.

Returns: nothing

Parameter	Type
`id`	box
`right`	num

`box.set_rightbottom(id, right, bottom)`

Moves the bottom-right corner.

Returns: nothing

Parameter	Type
`id`	box
`right`	num
`bottom`	num

`box.set_text(id, text)`

Changes the text.

Returns: nothing

Parameter	Type
`id`	box
`text`	string

`box.set_text_color(id, text_color)`

Changes the text color.

Returns: nothing

Parameter	Type
`id`	box
`text_color`	color

`box.set_top(id, top)`

Moves the top edge.

Returns: nothing

Parameter	Type
`id`	box
`top`	num

Tables

A grid of text pinned to a corner of the pane.

`table.cell(table_id, column, row, text="", width=0, height=0, text_color=na, text_halign="center", text_valign="center", text_size="normal", bgcolor=na, tooltip="")`

Sets a cell's text and colors.

Returns: nothing

Parameter	Type	Notes
`table_id`	table	
`column`	int	
`row`	int	
`text`	string	Default ""
`width`	num	Default 0
`height`	num	Default 0
`text_color`	color	Default na
`text_halign`	string	Default "center"
`text_valign`	string	Default "center"
`text_size`	string	Default "normal"
`bgcolor`	color	Default na
`tooltip`	string	Default ""

`table.cell_set_bgcolor(table_id, column, row, bgcolor)`

Changes a cell's background.

Returns: nothing

Parameter	Type
`table_id`	table
`column`	int
`row`	int
`bgcolor`	color

`table.cell_set_text(table_id, column, row, text)`

Changes a cell's text.

Returns: nothing

Parameter	Type
`table_id`	table
`column`	int
`row`	int
`text`	string

`table.cell_set_text_color(table_id, column, row, text_color)`

Changes a cell's text color.

Returns: nothing

Parameter	Type
`table_id`	table
`column`	int
`row`	int
`text_color`	color

`table.clear(table_id, start_column, start_row, end_column=na, end_row=na)`

Clears a range of cells.

Returns: nothing

Parameter	Type	Notes
`table_id`	table	
`start_column`	int	
`start_row`	int	
`end_column`	int	Default na
`end_row`	int	Default na

`table.delete(table_id)`

Removes the table.

Returns: nothing

Parameter	Type
`table_id`	table

`table.new(position, columns, rows, bgcolor=na, frame_color=na, frame_width=0, border_color=na, border_width=0)`

Creates a table fixed to a corner of the chart.

Returns: table

Parameter	Type	Notes
`position`	string	
`columns`	int	
`rows`	int	
`bgcolor`	color	Default na
`frame_color`	color	Default na
`frame_width`	int	Default 0
`border_color`	color	Default na
`border_width`	int	Default 0

Alerts

Rules and messages that notify you.

`alert(message, freq="once_per_bar")`

Fires an alert from code.

Returns: nothing

Parameter	Type	Notes
`message`	string	
`freq`	string	Default "once_per_bar"

`alertcondition(condition, title="", message="")`

An alert the member can pick in the alert dialog. Call at the top level of the script.

Returns: nothing

Parameter	Type	Notes
`condition`	bool	
`title`	string	Default ""
`message`	string	Default ""

Other symbols and timeframes

Read values from another timeframe or symbol. Never looks ahead.

`request.security(symbol, timeframe, expression, gaps=false, lookahead=false)`

The value of an expression on another symbol or a higher timeframe (only closed bars, never future data). Call at the top level of the script.

Returns: arg2

Parameter	Type	Notes
`symbol`	string	
`timeframe`	string	
`expression`	any	
`gaps`	bool	Default false
`lookahead`	bool	Default false

Order book (book)

Level 2 data: resting orders. Follows your plan like the charts do. Where there is no depth feed you get na and one notice.

Plan: Pro data. Returns na (plus one notice) when your plan does not include it.

`book.ask(level=0)`

Ask price at a book level (0 = best ask). Needs book data on your plan.

Returns: a number

Parameter	Type	Notes
`level`	int	Default 0

`book.asksize(level=0)`

Size resting at an ask level. Needs book data on your plan.

Returns: a number

Parameter	Type	Notes
`level`	int	Default 0

`book.bid(level=0)`

Bid price at a book level (0 = best bid). Needs book data on your plan.

Returns: a number

Parameter	Type	Notes
`level`	int	Default 0

`book.bidsize(level=0)`

Size resting at a bid level. Needs book data on your plan.

Returns: a number

Parameter	Type	Notes
`level`	int	Default 0

`book.depth(side, n=10)`

Total size of the first n levels of one side (book.side_bid or book.side_ask). Needs book data on your plan.

Returns: a number

Parameter	Type	Notes
`side`	string	
`n`	int	Default 10

`book.heat(price)`

Size resting at a price (the nearest book level). Needs book data on your plan.

Returns: a number

Parameter	Type
`price`	num

`book.imbalance(n=10)`

(bid depth - ask depth) / (bid depth + ask depth) over n levels, -1 to 1. Needs book data on your plan.

Returns: a number

Parameter	Type	Notes
`n`	int	Default 10

`book.wall(side, minSize)`

Price of the nearest level on one side with at least minSize resting (na if none). Needs book data on your plan.

Returns: a number

Parameter	Type
`side`	string
`minSize`	num

Order flow (flow)

Trades, delta, big trades, footprint and value area, per bar.

Plan: Pro data. Returns na (plus one notice) when your plan does not include it.

`flow.bigtrades(minSize)`

Number of trades in this bar of at least minSize. Needs flow data on your plan.

Returns: a number

Parameter	Type
`minSize`	num

`flow.bigtrades_vol(minSize)`

Total size of the trades in this bar of at least minSize. Needs flow data on your plan.

Returns: a number

Parameter	Type
`minSize`	num

`flow.buyvol`

Volume bought at the ask in this bar. Needs flow data on your plan.

Returns: a number

`flow.cvd`

Running total of delta (cumulative volume delta). Needs flow data on your plan.

Returns: a number

`flow.delta`

Buy volume minus sell volume in this bar. Needs flow data on your plan.

Returns: a number

`flow.footprint(price, side="both")`

Volume traded at a price in this bar (flow.side_buy, flow.side_sell or both). Needs flow data on your plan.

Returns: a number

Parameter	Type	Notes
`price`	num	
`side`	string	Default "both"

`flow.poc`

This bar's point of control (price with the most volume). Needs flow data on your plan.

Returns: a number

`flow.sellvol`

Volume sold at the bid in this bar. Needs flow data on your plan.

Returns: a number

`flow.trades`

Number of trades in this bar. Needs flow data on your plan.

Returns: a number

`flow.vah`

This bar's value area high (70% of volume). Needs flow data on your plan.

Returns: a number

`flow.val`

This bar's value area low (70% of volume). Needs flow data on your plan.

Returns: a number

GEX levels (gex)

Option gamma levels from GEX+. Updated a few minutes at a time, not an instant feed.

Plan: GEX+ data. Returns na (plus one notice) when your plan does not include it.

`gex.callwall`

Call wall level. Needs gex data on your plan.

Returns: a number

`gex.flip`

Gamma flip level. Needs gex data on your plan.

Returns: a number

`gex.level(n=0)`

The n-th strongest GEX level (0 = strongest). Needs gex data on your plan.

Returns: a number

Parameter	Type	Notes
`n`	int	Default 0

`gex.net`

Net gamma exposure. Needs gex data on your plan.

Returns: a number

`gex.putwall`

Put wall level. Needs gex data on your plan.

Returns: a number

`gex.updated`

Time the GEX levels were computed (ms). Needs gex data on your plan.

Returns: a whole number

`gex.zero_lower`

Lower zero-gamma level. Needs gex data on your plan.

Returns: a number

`gex.zero_upper`

Upper zero-gamma level. Needs gex data on your plan.

Returns: a number

ZoroOptions (opt)

Gamma nodes, ranges, IV rank and more. Updated a few minutes at a time, not an instant feed.

Plan: Pro data. Returns na (plus one notice) when your plan does not include it.

`opt.ivrank`

IV rank, 0 to 100. Needs opt data on your plan.

Returns: a number

`opt.maxpain`

Max pain strike. Needs opt data on your plan.

Returns: a number

`opt.node(n=0)`

Price of the n-th gamma node (0 = strongest). Needs opt data on your plan.

Returns: a number

Parameter	Type	Notes
`n`	int	Default 0

`opt.pc`

Put/call ratio. Needs opt data on your plan.

Returns: a number

`opt.range(kind="day")`

Low and high of an options range ("day", "week", "expected"). Needs opt data on your plan.

Returns: several values: [float,float]

Parameter	Type	Notes
`kind`	string	Default "day"

`opt.updated`

Time of the options data (ms); show it as "updated X min ago". Needs opt data on your plan.

Returns: a whole number

`opt.zone`

Current options zone name. Needs opt data on your plan.

Returns: text

Built-in variables

Values that exist in every script, such as close, bar_index and barstate.islast.

`bar_index`

Bar number, 0 for the first bar.

Returns: a whole number

`barstate.isconfirmed`

True when the bar is closed.

Returns: true or false

`barstate.isfirst`

True on the first bar.

Returns: true or false

`barstate.ishistory`

True on history bars.

Returns: true or false

`barstate.islast`

True on the last bar.

Returns: true or false

`barstate.islastconfirmedhistory`

True on the last closed history bar.

Returns: true or false

`barstate.isnew`

True on the first update of a bar.

Returns: true or false

`barstate.isrealtime`

True on live bars.

Returns: true or false

`close`

Close price of the bar (the latest price on the live bar).

Returns: a number

`dayofmonth`

Day of the month (UTC).

Returns: a whole number

`dayofweek`

Day of the week, 1 = Sunday ... 7 = Saturday (UTC).

Returns: a whole number

`high`

High price of the bar.

Returns: a number

`hl2`

(high + low) / 2.

Returns: a number

`hlc3`

(high + low + close) / 3.

Returns: a number

`hlcc4`

(high + low + close + close) / 4.

Returns: a number

`hour`

Hour (UTC).

Returns: a whole number

`last_bar_index`

Bar number of the last loaded bar.

Returns: a whole number

`low`

Low price of the bar.

Returns: a number

`minute`

Minute.

Returns: a whole number

`month`

Month of the bar, 1-12 (UTC).

Returns: a whole number

`ohlc4`

(open + high + low + close) / 4.

Returns: a number

`open`

Open price of the bar.

Returns: a number

`second`

Second.

Returns: a whole number

`syminfo.currency`

Quote currency.

Returns: text

`syminfo.mintick`

Tick size.

Returns: a number

`syminfo.pointvalue`

Value of one point move for one contract.

Returns: a number

`syminfo.ticker`

Symbol without exchange, e.g. "NQ".

Returns: text

`syminfo.tickerid`

Full symbol id.

Returns: text

`syminfo.type`

Market type: "futures", "crypto", "stock", "forex", "index".

Returns: text

`time`

Bar open time, ms since 1970 (UTC).

Returns: a whole number

`time_close`

Bar close time, ms since 1970 (UTC).

Returns: a whole number

`timeframe.isdaily`

True on daily charts.

Returns: true or false

`timeframe.isintraday`

True for minute and second timeframes.

Returns: true or false

`timeframe.isminutes`

True for minute timeframes.

Returns: true or false

`timeframe.ismonthly`

True on monthly charts.

Returns: true or false

`timeframe.isseconds`

True for second timeframes.

Returns: true or false

`timeframe.isweekly`

True on weekly charts.

Returns: true or false

`timeframe.multiplier`

Number part of the timeframe.

Returns: a whole number

`timeframe.period`

Chart timeframe, e.g. "5", "60", "D".

Returns: text

`volume`

Volume of the bar.

Returns: a number

`weekofyear`

Week number of the year (ISO).

Returns: a whole number

`year`

Year of the bar (UTC).

Returns: a whole number

Named constants

Ready-made names for colors, styles, shapes, sizes and positions.

Group	Names
`color`	`color.aqua` `color.black` `color.blue` `color.fuchsia` `color.gray` `color.green` `color.lime` `color.maroon` `color.navy` `color.olive` `color.orange` `color.purple` `color.red` `color.silver` `color.teal` `color.white` `color.yellow` `color.bull` `color.bear` `color.neutral` `color.accent`
`plot`	`plot.style_line` `plot.style_stepline` `plot.style_histogram` `plot.style_columns` `plot.style_circles` `plot.style_cross` `plot.style_area` `plot.style_areabr` `plot.style_linebr` `plot.style_stepline_diamond`
`shape`	`shape.xcross` `shape.cross` `shape.circle` `shape.triangleup` `shape.triangledown` `shape.flag` `shape.arrowup` `shape.arrowdown` `shape.labelup` `shape.labeldown` `shape.square` `shape.diamond`
`location`	`location.abovebar` `location.belowbar` `location.top` `location.bottom` `location.absolute`
`size`	`size.auto` `size.tiny` `size.small` `size.normal` `size.large` `size.huge`
`hline`	`hline.style_solid` `hline.style_dotted` `hline.style_dashed`
`line`	`line.style_solid` `line.style_dotted` `line.style_dashed` `line.style_arrow_left` `line.style_arrow_right` `line.style_arrow_both`
`label`	`label.style_none` `label.style_label_up` `label.style_label_down` `label.style_label_left` `label.style_label_right` `label.style_label_center` `label.style_label_upper_left` `label.style_label_upper_right` `label.style_label_lower_left` `label.style_label_lower_right` `label.style_circle` `label.style_square` `label.style_diamond`

Group	Names
	`label.style_arrowup` `label.style_arrowdown` `label.style_triangleup` `label.style_triangledown` `label.style_xcross` `label.style_cross` `label.style_flag` `label.style_text_outline`
`xloc`	`xloc.bar_index` `xloc.bar_time`
`yloc`	`yloc.price` `yloc.abovebar` `yloc.belowbar`
`extend`	`extend.none` `extend.left` `extend.right` `extend.both`
`text`	`text.align_left` `text.align_center` `text.align_right` `text.align_top` `text.align_bottom`
`position`	`position.top_left` `position.top_center` `position.top_right` `position.middle_left` `position.middle_center` `position.middle_right` `position.bottom_left` `position.bottom_center` `position.bottom_right`
`display`	`display.all` `display.none` `display.pane` `display.data_window` `display.price_scale` `display.status_line`
`format`	`format.price` `format.volume` `format.percent` `format.inherit`
`order`	`order.ascending` `order.descending`
`alert`	`alert.freq_all` `alert.freq_once_per_bar` `alert.freq_once_per_bar_close`
`scale`	`scale.right` `scale.left` `scale.none`
`book`	`book.side_bid` `book.side_ask`
`flow`	`flow.side_buy` `flow.side_sell` `flow.side_both`
`font`	`font.family_default` `font.family_monospace`
`barmerge`	`barmerge.gaps_on` `barmerge.gaps_off` `barmerge.lookahead_on` `barmerge.lookahead_off`

Group	Names
`dayofweek`	`dayofweek.sunday` `dayofweek.monday` `dayofweek.tuesday` `dayofweek.wednesday` `dayofweek.thursday` `dayofweek.friday` `dayofweek.saturday`

Keywords: `and`, `or`, `not`, `if`, `else`, `for`, `to`, `by`, `in`, `while`, `switch`, `var`, `varip`, `true`, `false`, `na`, `break`, `continue`.

Types: `int`, `float`, `bool`, `string`, `color`, `label`, `line`, `box`, `table`, `array`, `map`, `series`, `simple`, `const`.

Part 3. Cookbook

Simple moving average

The classic smooth line.

```
//@zscript 1
indicator("SMA", overlay=true)
len = input.int(20, "Length", minval=1)
plot(ta.sma(close, len), "SMA", color=#03c988, linewidth=2)
```



Sample bars, not market data. SMA.

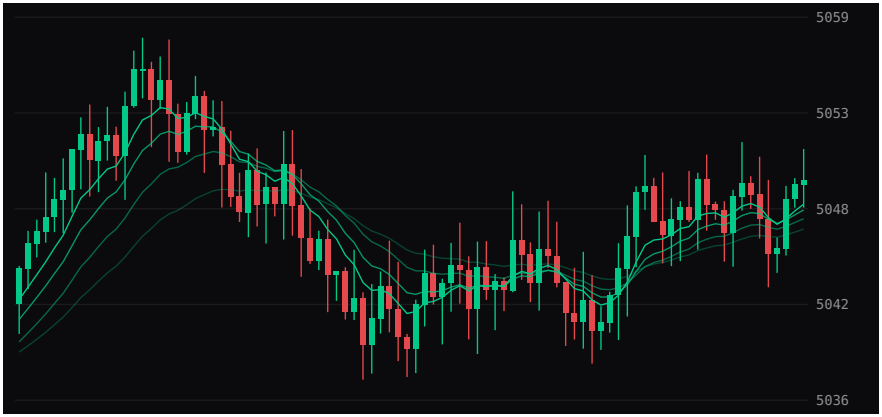
How it works

The average of the last `len` closes. Longer length, smoother and slower line. Try: switch `close` for `hl2` to average the middle of each bar instead.

EMA ribbon

Four averages that fan out in a trend.

```
//@zscript 1
indicator("EMA ribbon", overlay=true)
plot(ta.ema(close, 8), "8", color=color.new(#03c988, 0))
plot(ta.ema(close, 13), "13", color=color.new(#03c988, 25))
plot(ta.ema(close, 21), "21", color=color.new(#03c988, 50))
plot(ta.ema(close, 34), "34", color=color.new(#03c988, 70))
```



Sample bars, not market data. EMA ribbon.

How it works

Each line is an exponential average. When they spread apart, price has been moving one way for a while. When they tangle, it has not. The fading color helps you tell the lines apart.

Average cross marks

Arrows when a fast line crosses a slow one.

```
//@zscript 1
indicator("Cross marks", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast, "Fast", color=color.green)
plot(slow, "Slow", color=color.red)
plotshape(ta.crossover(fast, slow), "Up", shape.triangleup,
location.belowbar, color.green)
plotshape(ta.crossunder(fast, slow), "Down", shape.triangledown,
location.abovebar, color.red)
```



Sample bars, not market data. Cross marks.

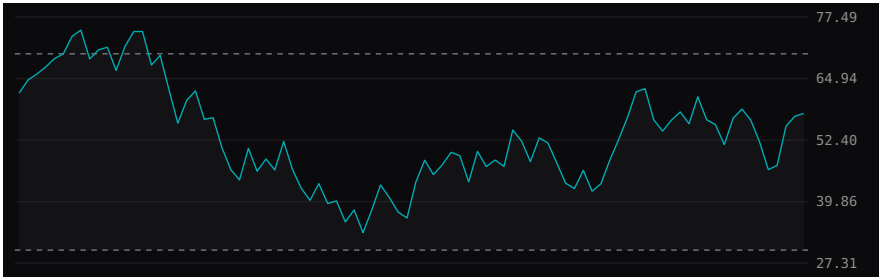
How it works

`ta.crossover(a, b)` is true on the bar where `a` moves from below `b` to above it. This is a teaching sample of a rule, not a trading system.

RSI with zones

RSI with shaded 30 and 70 lines.

```
//@zscript 1
indicator("RSI zones")
len = input.int(14, "Length", minval=2)
r = ta.rsi(close, len)
top = hline(70)
bot = hline(30)
fill(top, bot, color.new(color.gray, 92))
plot(r, "RSI", color=#00adb5)
```



Sample bars, not market data. RSI zones.

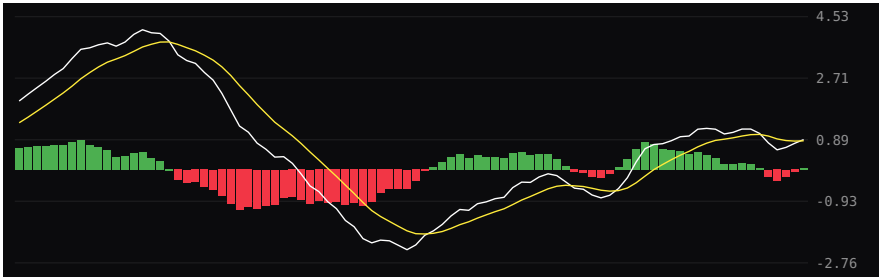
How it works

RSI measures how strong recent up moves are compared with down moves, on a 0 to 100 scale. 70 and 30 are common reference lines, not signals on their own.

MACD with colored histogram

Momentum from two averages.

```
//@zscript 1
indicator("MACD")
[macdLine, signal, hist] = ta.macd(close, 12, 26, 9)
plot(hist, "Histogram", style=plot.style_columns, color=hist >=
0 ? color.green : color.red)
plot(macdLine, "MACD", color=color.white)
plot(signal, "Signal", color=color.yellow)
```



Sample bars, not market data. MACD.

How it works

The histogram is the gap between the MACD line and its signal line. Green above zero, red below.

Bollinger Bands with shading

An average and a band of standard deviations.

```
//@zscript 1
indicator("Bands", overlay=true)
len = input.int(20, "Length")
mult = input.float(2.0, "Width", step=0.1)
[mid, up, lo] = ta.bb(close, len, mult)
p1 = plot(up, "Upper", color=#00adb5)
p2 = plot(lo, "Lower", color=#00adb5)
plot(mid, "Middle", color=color.gray)
fill(p1, p2, color.new(#00adb5, 92))
```



Sample bars, not market data. Bands.

How it works

`ta.bb` returns three values at once, so we take them apart with square brackets.

Keltner channel

An average with bands from average true range.

```
//@zscript 1
indicator("Keltner", overlay=true)
[mid, up, lo] = ta.kc(close, 20, 1.5)
p1 = plot(up, "Upper", color=color.orange)
p2 = plot(lo, "Lower", color=color.orange)
plot(mid, "Middle", color=color.gray)
fill(p1, p2, color.new(color.orange, 93))
```



Sample bars, not market data. Keltner.

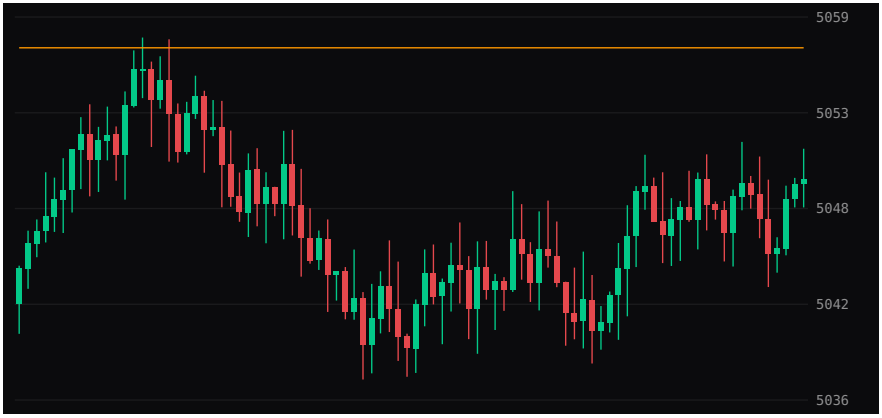
How it works

Same idea as Bollinger Bands, but the width follows the typical bar size (ATR) rather than standard deviation.

ATR trailing line

A line that keeps its distance from price.

```
//@zscript 1
indicator("ATR trail", overlay=true)
mult = input.float(3.0, "ATR multiple", step=0.5)
atr1 = ta.atr(14)
var float trail = na
raw = close - atr1 * mult
trail := na(trail) ? raw : math.max(trail, raw)
plot(trail, "Trail", color=color.orange, style=plot.style_stepline)
```



Sample bars, not market data. ATR trail.

How it works

The line only moves up, never down, so it traces a path that stays a set number of bar-sizes below the close. A teaching example of `var`.

Supertrend

A built-in trend line that flips color.

```
//@zscript 1
indicator("Supertrend", overlay=true)
[st, dir] = ta.supertrend(3.0, 10)
plot(st, "Supertrend", color=dir < 0 ? color.green : color.red,
linewidth=2)
```



Sample bars, not market data. Supertrend.

How it works

`dir` is -1 in an up-trend and 1 in a down-trend. We use it to choose the color.

Session VWAP

Average price weighted by volume.

```
//@zscript 1
indicator("VWAP", overlay=true)
plot(ta.vwap(hlc3), "VWAP", color=#03c988, linewidth=2)
```



Sample bars, not market data. VWAP.

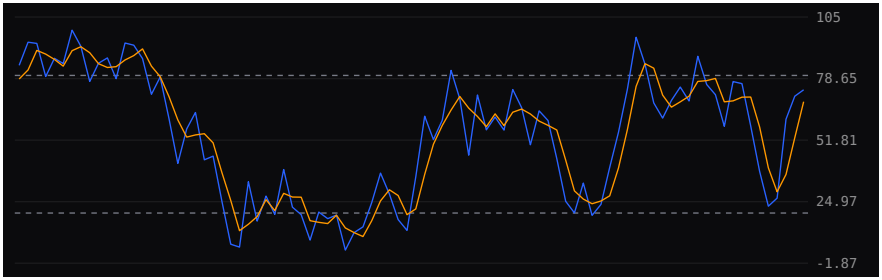
How it works

`ta.vwap` weights each bar's typical price by its volume and starts again each day (UTC). Pass `anchor="W"` or `"M"` for a weekly or monthly start.

Stochastic

Where the close sits inside the recent range.

```
//@zscript 1
indicator("Stochastic")
k = ta.stoch(close, high, low, 14)
d = ta.sma(k, 3)
hline(80)
hline(20)
plot(k, "%K", color=color.blue)
plot(d, "%D", color=color.orange)
```



Sample bars, not market data. Stochastic.

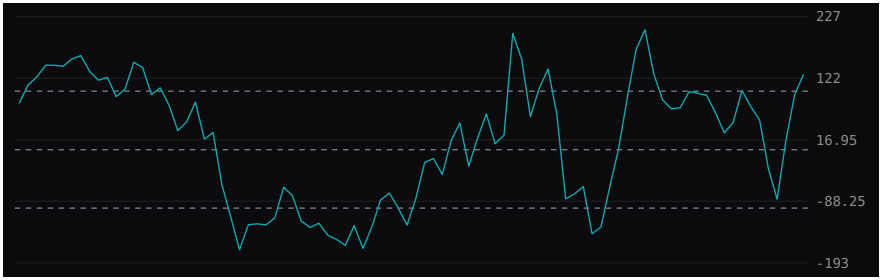
How it works

100 means the close is at the top of the last 14 bars' range, 0 the bottom.

CCI

How far typical price strays from its average.

```
//@zscript 1
indicator("CCI")
c = ta.cci(hlc3, 20)
hline(100)
hline(-100)
hline(0)
plot(c, "CCI", color=#00adb5)
```



Sample bars, not market data. CCI.

How it works

Bigger than 100 or smaller than -100 means an unusually big move compared with recent bars.

On-balance volume

A running volume total that follows direction.

```
//@zscript 1
indicator("OBV")
plot(ta.obv, "OBV", color=#03c988)
```



Sample bars, not market data. OBV.

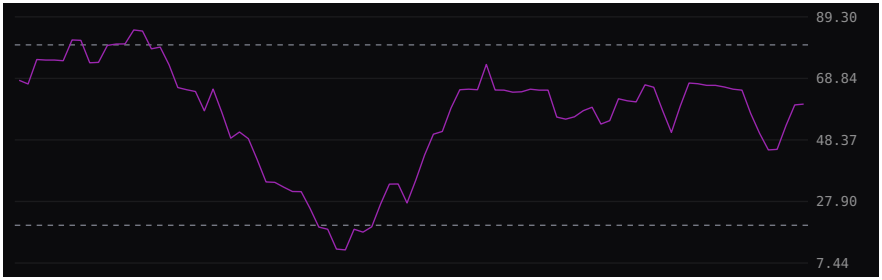
How it works

Volume is added on up closes and subtracted on down closes. Look at the shape, not the number.

Money flow index

RSI that includes volume.

```
//@zscript 1
indicator("MFI")
hline(80)
hline(20)
plot(ta.mfi(hlc3, 14), "MFI", color=color.purple)
```



Sample bars, not market data. MFI.

How it works

Like RSI, but each bar counts more when volume was high.

Donchian channel

Highest high and lowest low of N bars.

```
//@zscript 1
indicator("Donchian", overlay=true)
n = input.int(20, "Length", minval=2)
hi = ta.highest(high, n)
lo = ta.lowest(low, n)
p1 = plot(hi, "High", color=color.green)
p2 = plot(lo, "Low", color=color.red)
fill(p1, p2, color.new(color.gray, 94))
```



Sample bars, not market data. Donchian.

How it works

A breakout channel: a close above the top line is a new N-bar high.

Swing high and low labels

Mark pivots after they are confirmed.

```
//@zscript 1
indicator("Pivots", overlay=true)
n = input.int(5, "Bars each side", minval=1, maxval=20)
ph = ta.pivothigh(high, n, n)
pl = ta.pivotlow(low, n, n)
if not na(ph)
    label.new(bar_index - n, ph, "H",
style=label.style_label_down, color=color.red,
textcolor=color.white)
if not na(pl)
    label.new(bar_index - n, pl, "L", style=label.style_label_up,
color=color.green, textcolor=color.white)
```



Sample bars, not market data. Pivots.

How it works

A pivot needs `n` bars on each side to be sure, so it is only known `n` bars later. That is why the label is placed at `bar_index - n`: it goes back to where the pivot really was, without ever using a future bar.

Previous day high and low

Lines from yesterday's range.

```
//@zscript 1
indicator("Prior day", overlay=true)
pdh = request.security(syminfo.tickerid, "D", high[1])
pdl = request.security(syminfo.tickerid, "D", low[1])
plot(pdh, "Prior high", color=color.green,
style=plot.style_stepline)
plot(pdl, "Prior low", color=color.red, style=plot.style_stepline)
```

How it works

`request.security` asks for a value from another timeframe. `[1]` on the daily series means "the last finished day", which keeps it free of look-ahead.

Higher timeframe trend

Show the 1-hour trend on a small chart.

```
//@zscript 1
indicator("HTF trend", overlay=true)
tf = input.timeframe("60", "Higher timeframe")
htf = request.security(syminfo.tickerid, tf, ta.ema(close, 50)[1])
plot(htf, "HTF EMA", color=color.yellow, style=plot.style_stepline)
bgcolor(close > htf ? color.new(color.green, 94) :
color.new(color.red, 94))
```

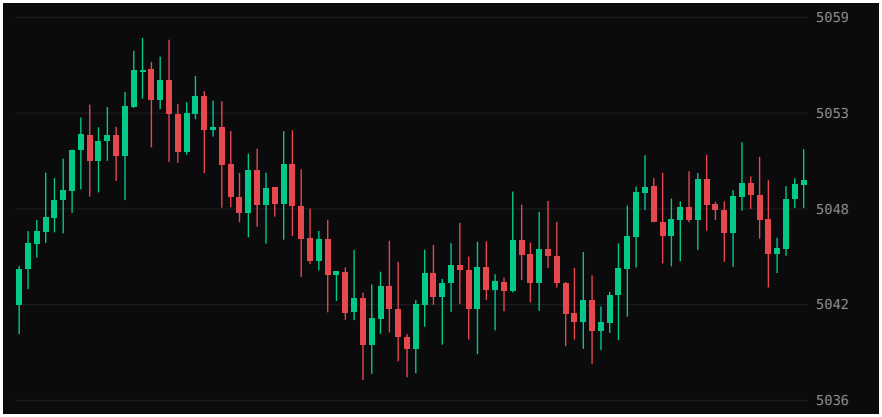
How it works

The background turns green when price is above the higher timeframe average.

Inside bars

Highlight bars within the previous bar's range.

```
//@zscript 1
indicator("Inside bar", overlay=true)
inside = high <= high[1] and low >= low[1]
barcolor(inside ? color.yellow : na)
```



Sample bars, not market data. Inside bar.

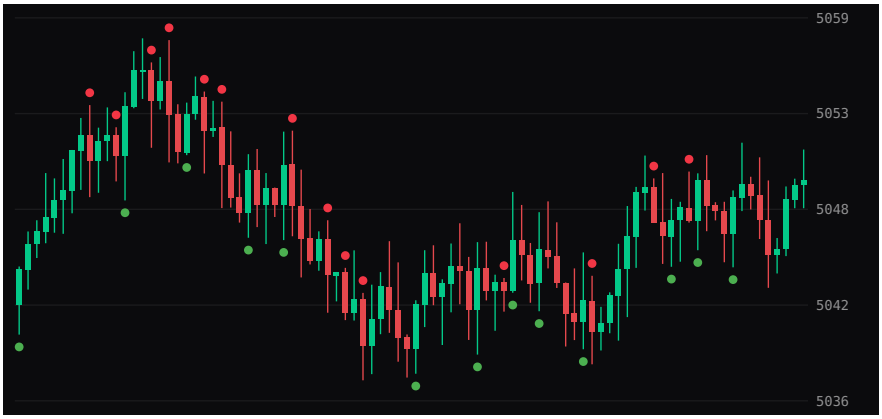
How it works

Returning `na` for a color leaves the candle alone.

Engulfing candles

Mark bars that wrap the previous body.

```
//@zscript 1
indicator("Engulfing", overlay=true)
bull = close > open and close[1] < open[1] and close >= open[1]
and open <= close[1]
bear = close < open and close[1] > open[1] and close <= open[1]
and open >= close[1]
plotshape(bull, "Bullish", shape.arrowup, location.belowbar,
color.green)
plotshape(bear, "Bearish", shape.arrowdown, location.abovebar,
color.red)
```



Sample bars, not market data. Engulfing.

How it works

A pattern finder in four lines. A pattern is a shape, not a prediction.

Fair value gaps

Box the gap between candle 1 and candle 3.

```
//@zscript 1
indicator("FVG", overlay=true, max_bars_back=5)
bullGap = low > high[2]
bearGap = high < low[2]
if bullGap
    box.new(bar_index - 2, low, bar_index + 10, high[2],
border_color=color.green, bgcolor=color.new(color.green, 88))
if bearGap
    box.new(bar_index - 2, low[2], bar_index + 10, high,
border_color=color.red, bgcolor=color.new(color.red, 88))
```



Sample bars, not market data. FVG.

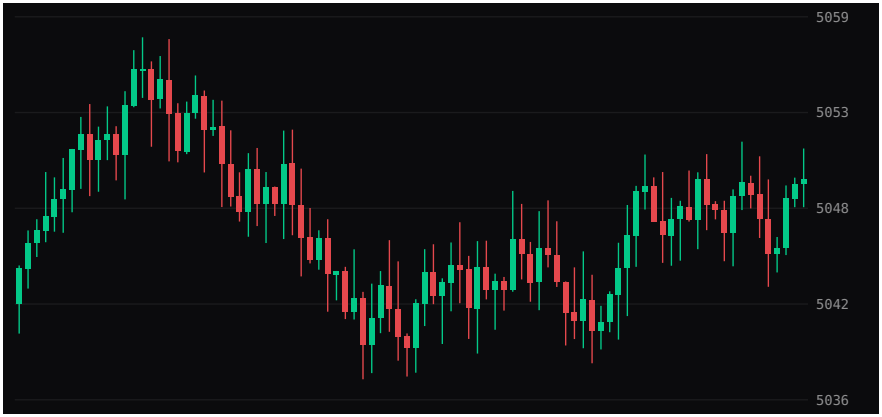
How it works

A gap exists when the wicks of candles one and three do not overlap. We draw a box from the gap forward ten bars.

Opening range box

Box the first bars of the session.

```
//@zscript 1
indicator("Opening range", overlay=true)
bars = input.int(6, "Bars in range", minval=1)
newDay = dayofmonth != dayofmonth[1]
var int count = 0
var float hi = na
var float lo = na
var box b = na
if newDay
    count := 1
    hi := high
    lo := low
    b := box.new(bar_index, high, bar_index, low,
border_color=color.gray, bgcolor=color.new(color.gray, 90))
else if count < bars
    count += 1
    hi := math.max(hi, high)
    lo := math.min(lo, low)
    box.set_right(b, bar_index)
    box.set_top(b, hi)
    box.set_bottom(b, lo)
```



Sample bars, not market data. Opening range.

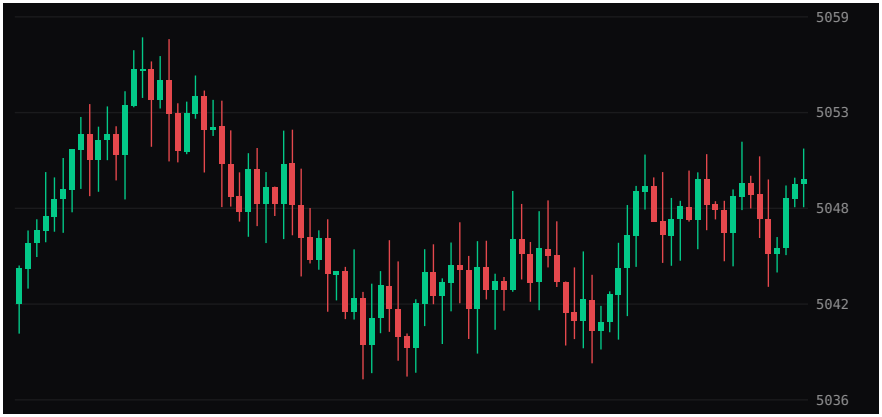
How it works

Each new day starts a fresh box. For the first `bars` bars it grows to include every high and low.

Session background

Tint the background by time of day (UTC).

```
//@zscript 1
indicator("Session tint", overlay=true)
minuteOfDay = hour * 60 + minute
inSession = minuteOfDay >= 13 * 60 + 30 and minuteOfDay < 20 * 60
bgcolor(inSession ? color.new(color.blue, 94) : na)
```



Sample bars, not market data. Session tint.

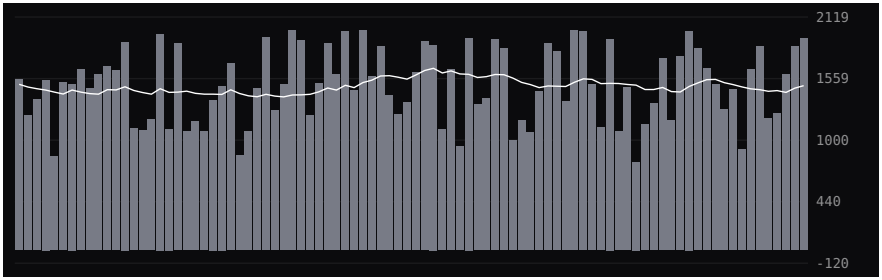
How it works

`hour` and `minute` are in UTC in Zscript v1, so the window above (13:30 to 20:00 UTC) is the New York cash session in summer. Move the two numbers for other markets or for winter time.

Volume spike marks

Mark bars with unusually high volume.

```
//@zscript 1
indicator("Volume spikes")
mult = input.float(2.0, "Times the average", step=0.1)
avg = ta.sma(volume, 20)
spike = volume > avg * mult
plot(volume, "Volume", style=plot.style_columns, color=spike ?
color.orange : color.gray)
plot(avg, "Average", color=color.white)
```



Sample bars, not market data. Volume spikes.

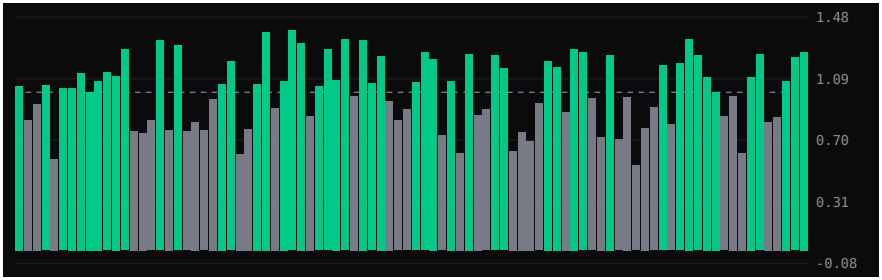
How it works

Orange columns are bars where volume was more than `mult` times its 20-bar average.

Relative volume

Volume as a multiple of normal.

```
//@zscript 1
indicator("Relative volume")
rv = volume / ta.sma(volume, 30)
hline(1)
plot(rv, "Relative volume", style=plot.style_columns, color=rv >
1 ? #03c988 : color.gray)
```



Sample bars, not market data. Relative volume.

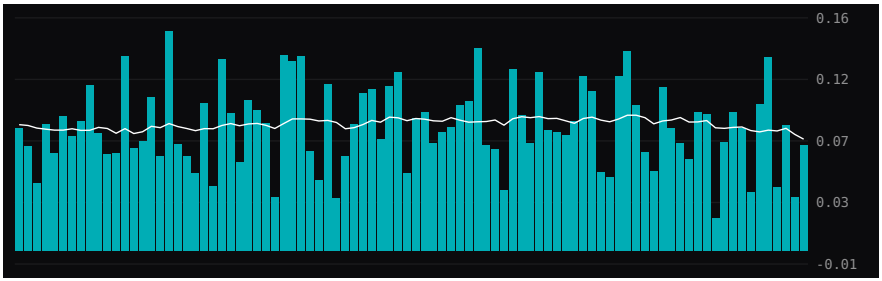
How it works

1.0 is normal. 2.0 means twice the 30-bar average.

Bar range as percent

How big is each bar, relative to price?

```
//@zscript 1
indicator("Range %")
pct = (high - low) / close * 100
plot(pct, "Range %", style=plot.style_columns, color=#00adb5)
plot(ta.sma(pct, 20), "Average", color=color.white)
```



Sample bars, not market data. Range %.

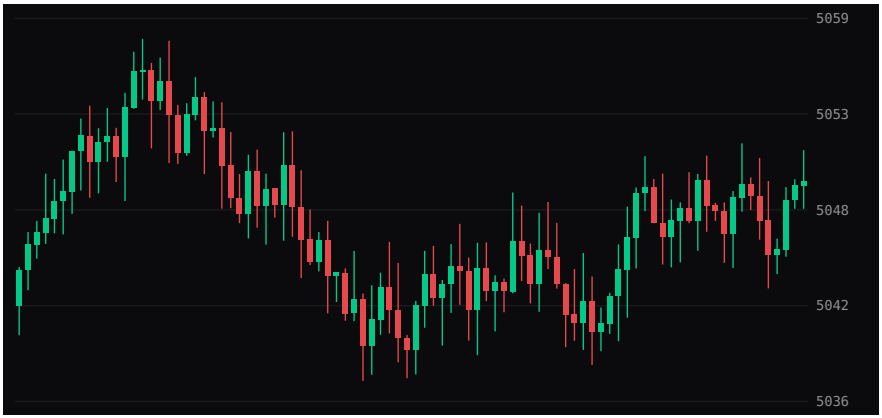
How it works

Useful for comparing quiet and busy markets on the same scale.

Opening gap marks

Mark days that open away from the prior close.

```
//@zscript 1
indicator("Gap marks", overlay=true)
minGap = input.float(0.3, "Minimum gap %", step=0.1)
newDay = dayofmonth != dayofmonth[1]
gapPct = (open - close[1]) / close[1] * 100
plotshape(newDay and gapPct > minGap, "Gap up", shape.triangleup,
location.belowbar, color.green)
plotshape(newDay and gapPct < -minGap, "Gap down",
shape.triangledown, location.abovebar, color.red)
```



Sample bars, not market data. Gap marks.

How it works

Compares each day's first open with the last close before it.

Label the day's high

Write the high of the day on the newest bar.

```
//@zscript 1
indicator("Day high label", overlay=true)
var float dayHigh = na
if dayofmonth != dayofmonth[1]
    dayHigh := high
else
    dayHigh := math.max(dayHigh, high)
if barstate.islast
    label.new(bar_index, dayHigh, "Day high " +
str.tostring(dayHigh, "#.##"), style=label.style_label_left)
```



Sample bars, not market data. Day high label.

How it works

Drawing only on `barstate.islast` keeps the script light: one label, not thousands.

Info table

A small table with key numbers.

```
//@zscript 1
indicator("Info table", overlay=true)
var t = table.new(position.top_right, 2, 3)
if barstate.islast
    table.cell(t, 0, 0, "ATR(14)")
    table.cell(t, 1, 0, str.tostring(ta.atr(14), "#.##"))
    table.cell(t, 0, 1, "RSI(14)")
    table.cell(t, 1, 1, str.tostring(ta.rsi(close, 14), "#.#"))
    table.cell(t, 0, 2, "Volume")
    table.cell(t, 1, 2, str.tostring(volume, "#"))
```



Sample bars, not market data. Info table.

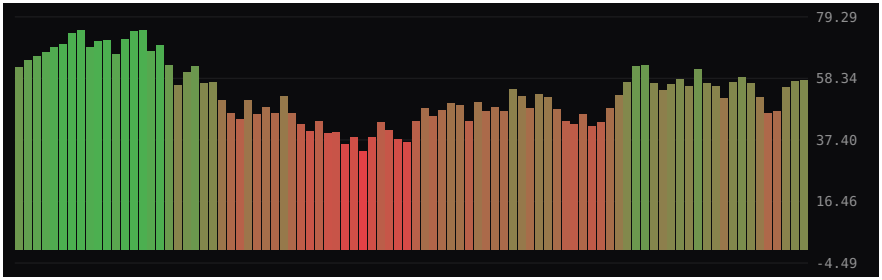
How it works

The indicator functions are called on every bar (as they must be) and the table is only written on the last bar.

Gradient volume

Color by how big a value is.

```
//@zscript 1
indicator("Gradient")
r = ta.rsi(close, 14)
c = color.from_gradient(r, 30, 70, color.red, color.green)
plot(r, "RSI", style=plot.style_columns, color=c)
```



Sample bars, not market data. Gradient.

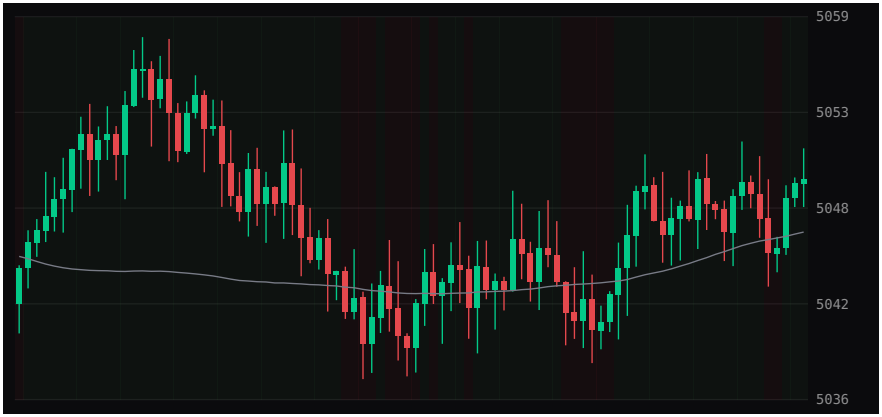
How it works

`color.from_gradient(value, low, high, colorLow, colorHigh)` blends between two colors.

Trend filter background

Green when above a long average, red when below.

```
//@zscript 1
indicator("Trend filter", overlay=true)
len = input.int(100, "Length")
a = ta.sma(close, len)
plot(a, "Average", color=color.gray)
bgcolor(close > a ? color.new(color.green, 94) :
color.new(color.red, 94))
```



Sample bars, not market data. Trend filter.

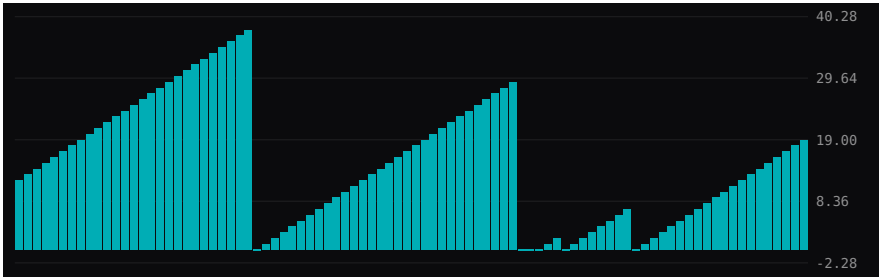
How it works

The simplest possible regime filter.

Bars since last cross

Count bars since a signal.

```
//@zscript 1
indicator("Bars since cross")
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
n = ta.barssince(ta.cross(fast, slow))
plot(n, "Bars since last cross", style=plot.style_columns,
color=#00adb5)
```



Sample bars, not market data. Bars since cross.

How it works

`ta.barssince(condition)` counts bars since the condition was last true.

Correlation with another symbol

How closely two markets move together.

```
//@zscript 1
indicator("Correlation")
other = input.string("NQ1!", "Other symbol")
o = request.security(other, timeframe.period, close)
c = ta.correlation(close, o, 50)
hline(0)
plot(c, "Correlation (50 bars)", color=#00adb5)
```

How it works

Near 1 they move together, near -1 they move opposite, near 0 they do not relate.

Regression line and bands

A fitted line through recent closes.

```
//@zscript 1
indicator("Regression", overlay=true)
len = input.int(50, "Length")
lr = ta.linreg(close, len, 0)
sd = ta.stdev(close, len)
p1 = plot(lr + 2 * sd, "Upper", color=color.gray)
p2 = plot(lr - 2 * sd, "Lower", color=color.gray)
plot(lr, "Line", color=#03c988)
fill(p1, p2, color.new(color.gray, 94))
```



Sample bars, not market data. Regression.

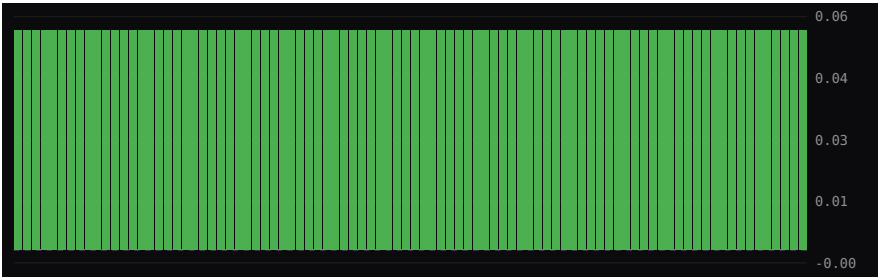
How it works

The fitted line and a band two standard deviations away.

Order book imbalance

Which side has more resting size (Pro data).

```
//@zscript 1
indicator("Book imbalance")
n = input.int(10, "Levels", minval=1, maxval=40)
imb = book.imbalance(n)
hline(0)
plot(imb, "Imbalance", style=plot.style_columns, color=imb >= 0 ?
color.green : color.red)
```



Sample bars, not market data. Book imbalance.

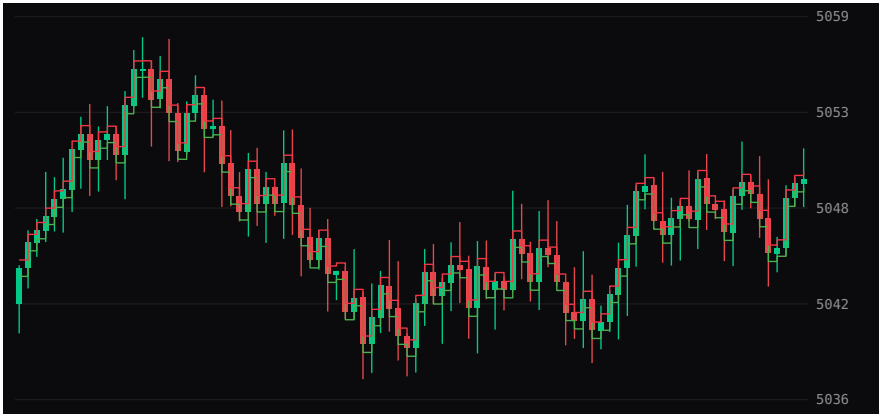
How it works

Above zero means more size resting on the bid side. Without a plan that includes the data, or on a symbol with no depth feed, this shows nothing and a one-time notice explains why.

Nearest book walls

Draw the nearest big resting levels (Pro data).

```
//@zscript 1
indicator("Book walls", overlay=true)
minSize = input.int(500, "Minimum size", minval=1)
plot(book.wall(book.side_bid, minSize), "Bid wall",
color=color.green, style=plot.style_stepline)
plot(book.wall(book.side_ask, minSize), "Ask wall",
color=color.red, style=plot.style_stepline)
```



Sample bars, not market data. Book walls.

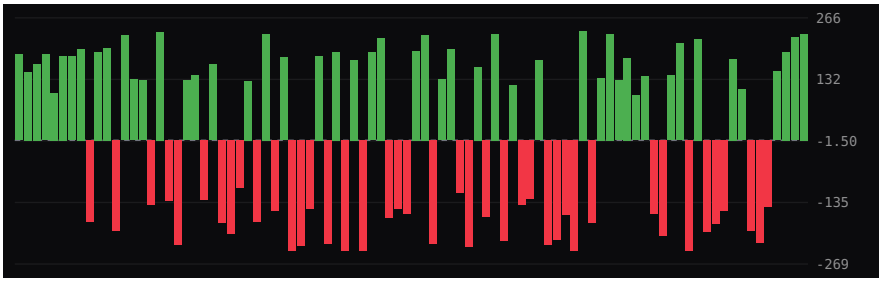
How it works

A wall is a single price level with at least `minSize` resting. Walls can be pulled at any time, so treat them as information, not a promise.

Delta columns

Buyers minus sellers per bar (Pro data).

```
//@zscript 1
indicator("Delta")
hline(0)
plot(flow.delta, "Delta", style=plot.style_columns,
color=flow.delta >= 0 ? color.green : color.red)
```



Sample bars, not market data. Delta.

How it works

Positive when more volume traded at the ask than at the bid.

Cumulative delta

Running total of delta (Pro data).

```
//@zscript 1
indicator("CVD")
plot(flow.cvd, "CVD", color=#00adb5)
```



Sample bars, not market data. CVD.

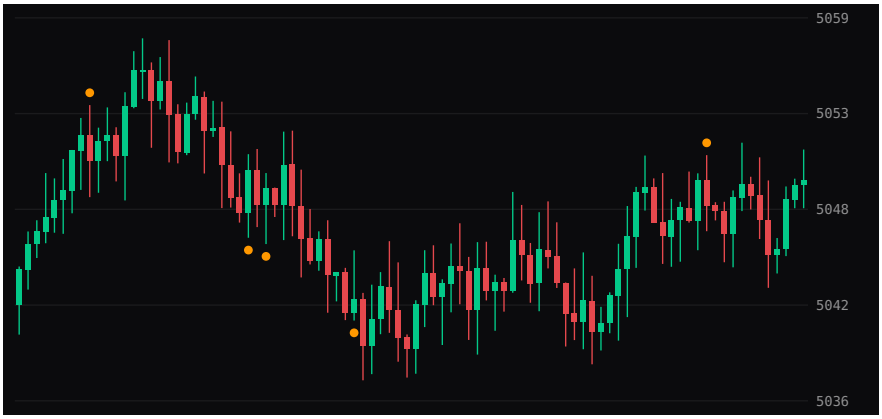
How it works

Compare its slope with price to see whether buying or selling is driving the move.

Delta against price

Mark bars where price and delta disagree (Pro data).

```
//@zscript 1
indicator("Delta divergence", overlay=true)
newHigh = high >= ta.highest(high, 20)
weak = flow.delta < 0
plotshape(newHigh and weak, "High on negative
delta", shape.xcross, location.abovebar, color.orange)
newLow = low <= ta.lowest(low, 20)
strong = flow.delta > 0
plotshape(newLow and strong, "Low on positive
delta", shape.xcross, location.belowbar, color.orange)
```



Sample bars, not market data. Delta divergence.

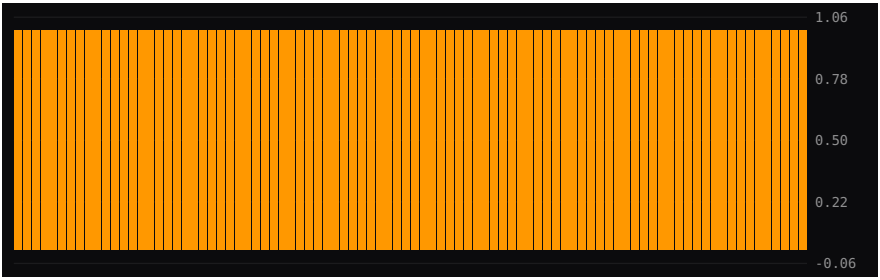
How it works

A new 20-bar high that traded with negative delta (or a low with positive delta) is flagged. It is a thing to look at, not a signal.

Big trade count

How many large single trades hit the bar (Pro data).

```
//@zscript 1
indicator("Big trades")
minSize = input.int(50, "Minimum trade size", minval=1)
n = flow.bigtrades(minSize)
plot(n, "Big trades", style=plot.style_columns, color=n > 0 ?
color.orange : color.gray)
```



Sample bars, not market data. Big trades.

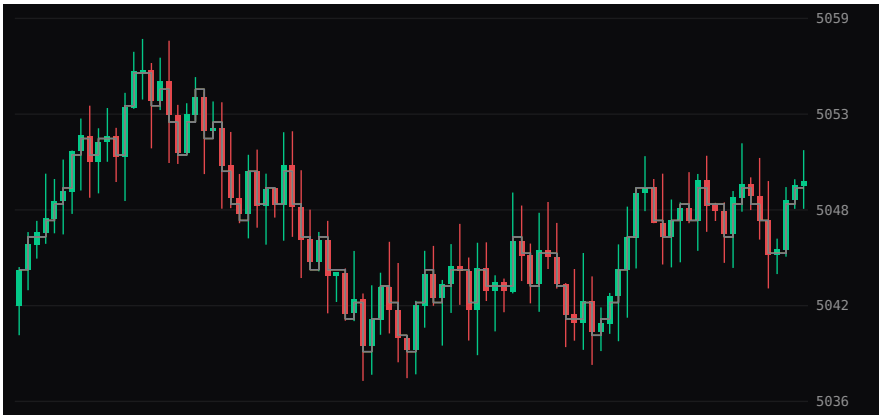
How it works

Counts single trades of at least `minSize` contracts or units.

POC and value area

Point of control with value area edges (Pro data).

```
//@zscript 1
indicator("POC and value area", overlay=true)
plot(flow.poc, "POC", color=color.yellow,
style=plot.style_stepline)
p1 = plot(flow.vah, "VAH", color=color.gray,
style=plot.style_stepline)
p2 = plot(flow.val, "VAL", color=color.gray,
style=plot.style_stepline)
fill(p1, p2, color.new(color.gray, 94))
```



Sample bars, not market data. POC and value area.

How it works

The price with the most volume in the bar, and the range holding most of the volume.

GEX levels

Call wall, put wall and flip as lines (GEX+ data).

```
//@zscript 1
indicator("GEX levels", overlay=true)
plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
plot(gex.flip, "Flip", color=color.yellow,
style=plot.style_stepline)
```



Sample bars, not market data. GEX levels.

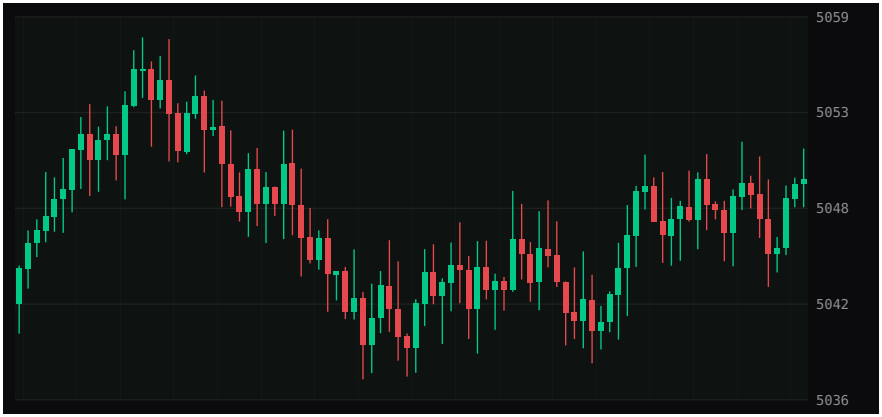
How it works

The levels come from the latest GEX+ update, not an instant feed. For ES and NQ they follow the GEX+ page mapping.

Above or below the flip

Tint the background by gamma regime (GEX+ data).

```
//@zscript 1
indicator("Flip regime", overlay=true)
f = gex.flip
bgcolor(na(f) ? na : close > f ? color.new(color.green, 94) :
color.new(color.red, 94))
```



Sample bars, not market data. Flip regime.

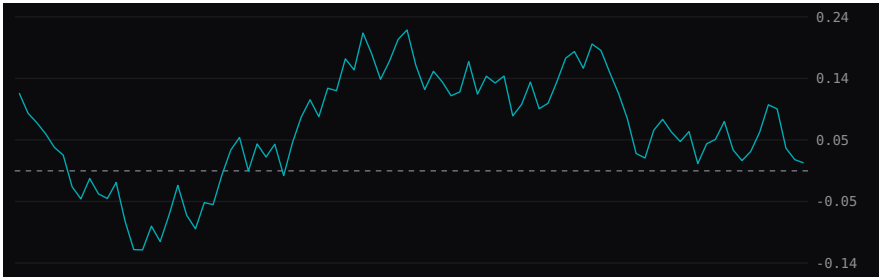
How it works

Green above the flip level, red below. No tint when there is no data.

Distance to the call wall

How far price is from a level, in percent (GEX+ data).

```
//@zscript 1
indicator("Distance to call wall")
d = (gex.callwall - close) / close * 100
hline(0)
plot(d, "% to call wall", color=#00adb5)
```



Sample bars, not market data. Distance to call wall.

How it works

Positive when price is below the wall.

Option range edges

The expected range as two lines (ZoroOptions data).

```
//@zscript 1
indicator("Option range", overlay=true)
[lo, hi] = opt.range("expected")
plot(hi, "Upper", color=color.gray, style=plot.style_stepline)
plot(lo, "Lower", color=color.gray, style=plot.style_stepline)
```



Sample bars, not market data. Option range.

How it works

Range numbers are updated a few minutes at a time, not tick by tick.

IV rank readout

A table with IV rank and put/call ratio (ZoroOptions data).

```
//@zscript 1
indicator("Options readout", overlay=true)
var t = table.new(position.top_right, 2, 2)
if barstate.islast
    table.cell(t, 0, 0, "IV rank")
    table.cell(t, 1, 0, str.tostring(opt.ivrank, "#.#"))
    table.cell(t, 0, 1, "Put/call")
    table.cell(t, 1, 1, str.tostring(opt.pc, "#.##"))
```



Sample bars, not market data. Options readout.

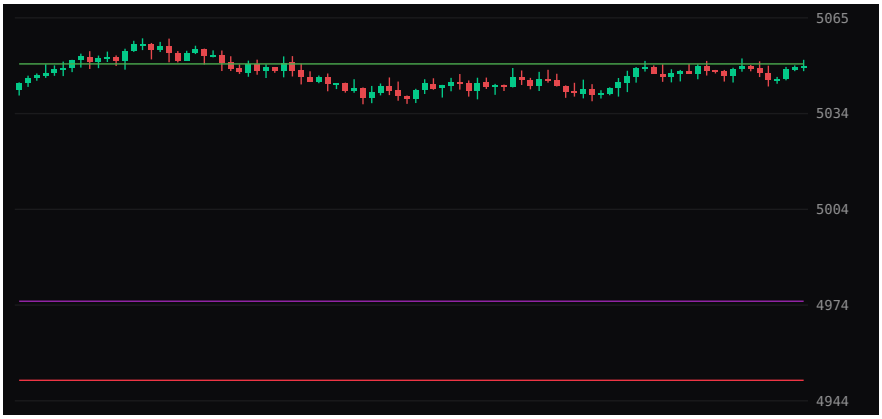
How it works

Shows "NaN" style blanks when your plan lacks the data; the script still runs.

Gamma nodes with GEX walls

Combine two data kinds in one script (GEX+ and ZoroOptions data).

```
//@zscript 1
indicator("Nodes and walls", overlay=true)
plot(opt.node(1), "Node 1", color=color.purple,
style=plot.style_stepline)
plot(opt.node(2), "Node 2", color=color.purple,
style=plot.style_stepline)
plot(gex.callwall, "Call wall", color=color.green,
style=plot.style_stepline)
plot(gex.putwall, "Put wall", color=color.red,
style=plot.style_stepline)
```



Sample bars, not market data. Nodes and walls.

How it works

Each data kind follows your plan on its own. If one is missing, only its lines go blank.

Cross alert

An alert you can set from the chart.

```
//@zscript 1
indicator("Cross alert", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast)
plot(slow)
alertcondition(ta.crossover(fast, slow), "Up cross", "Fast EMA
crossed above slow EMA")
alertcondition(ta.crossunder(fast, slow), "Down cross", "Fast EMA
crossed below slow EMA")
```

How it works

After you add the script, the alert dialog lists both rules.

Level alert with a message

Build the message from your own settings.

```
//@zscript 1
indicator("Level alert", overlay=true)
level = input.float(100.0, "Level")
hline(level)
if ta.crossover(close, level) and barstate.isconfirmed
    alert("Close above " + str.tostring(level, "#.##"))
```

How it works

The message is built from the number you chose in the settings.

Part 4. The course

Lesson 1. Meet Zscript

Zscript is the small language you use to build your own chart indicators on TradeZoro. You write a few lines, add them to the chart, and your idea is drawn on the candles. It is free for every member.

In this lesson you add your first script and learn what its four parts do.

```
//@zscript 1
indicator("My first line", overlay=true)
average = ta.sma(close, 20)
plot(average, title="20-bar average")
```

The four parts

- **Header:** `//@zscript 1` says this is Zscript, version 1.
- **Declaration:** `indicator(...)` names the script. `overlay=true` puts it on the price chart.
- **Logic:** `average = ta.sma(close, 20)` makes a value from the closing prices.
- **Drawing:** `plot(...)` shows it.

Press **Add to chart** and a line follows price. Change 20 to 50 and add it again: the line gets smoother. Changing a number and looking is how you learn.

Education only. Not financial advice. A line on a chart describes the past; it does not promise anything.

Exercises

1. **Add your first script.** Paste the script above into the editor and add it to a chart. Then change 20 to 5 and to 100. Write one sentence about how the line changes.
2. **Rename and recolor.** Change the title to your own name and add color=color.orange to the plot call. Add it again.

Lesson 2. Bars and series

A chart is a row of bars. Zscript runs your script once for every bar, oldest to newest. On each run, `close` means the close of the bar being worked on.

A value that has one number per bar is a **series**. To see an older value, use square brackets: `close[1]` is the close one bar ago.

```
//@zscript 1
indicator("Bar change")
change1 = close - close[1]
plot(change1, "Change", style=plot.style_histogram)
```

On the first bar there is nothing before it, so `close[1]` is **na**: not available. Math with na gives na, and plots skip it. Use `nz(x)` to swap na for zero, or `na(x)` to test for it.

You can only look back. Zscript has no way to read a bar that has not happened yet.

Exercises

1. **Two-bar change.** Plot close minus close[2] as a histogram. Which bars are missing at the left edge, and why?
2. **Safe change.** Use nz() so the first bar shows 0 instead of nothing.

Lesson 3. Values, types and colors

Every value has a type: `int` (whole numbers), `float` (decimals), `bool` (true or false), `string` (text) and `color`. Zscript usually works the type out for you.

`=` creates a value. `:=` changes one that already exists. `x ? a : b` picks `a` when x is true, else `b`.

```
//@zscript 1
indicator("Up or down volume")
up = close >= open
plot(volume, "Volume", color=up ? color.green : color.red,
style=plot.style_columns)
```

Colors: `#03c988`, `color.red`, or `color.new(base, transparency)` where 0 is solid and 100 invisible.

Exercises

1. **Three colors.** Color volume green on up bars, red on down bars, and gray when close equals open.
2. **Soft color.** Use color.new(...) to make the green 60 percent transparent.

Lesson 4. Moving averages and indicators

Most indicators are one function call. The `ta` namespace holds them: `ta.sma`, `ta.ema`, `ta.rsi`, `ta.atr`, `ta.macd` and many more.

```
//@zscript 1
indicator("Two averages", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plot(fast, color=color.green)
plot(slow, color=color.red)
```

Some functions give back several values. Take them apart with square brackets: `[mid, up, lo] = ta.bb(close, 20, 2.0)`.

Call `ta` functions on every bar, not inside an `if`. They keep a running memory, and skipping bars gives wrong answers.

Exercises

1. **Add RSI.** Build an indicator in its own pane: RSI(14) with hline(30) and hline(70).
2. **Bands.** Plot Bollinger Bands with ta.bb and shade between the outer lines using fill().

Lesson 5. Decisions and signals

A **condition** is a true or false value for each bar. `ta.crossover(a, b)` is true on the bar where a moves above b.

```
//@zscript 1
indicator("Cross marks", overlay=true)
fast = ta.ema(close, 9)
slow = ta.ema(close, 21)
plotshape(ta.crossover(fast, slow), "Up", shape.triangleup,
location.belowbar, color.green)
plotshape(ta.crossunder(fast, slow), "Down", shape.triangledown,
location.abovebar, color.red)
```

Use `if` / `else` to choose, and `and`, `or`, `not` to combine conditions. This lesson shows a rule on a chart. It is a teaching sample, not a trading system.

Exercises

1. **Add a filter.** Only mark an up cross when close is also above a 100-bar SMA.
2. **Color the pane.** Use bgcolor to tint the background green while fast is above slow.

Lesson 6. Remembering with var

Normal values start fresh on every bar. A `var` value is set once and keeps what you give it.

```
//@zscript 1
indicator("Count green bars")
var int greens = 0
if close > open
    greens := greens + 1
plot(greens)
```

Use `var` for running totals, the last signal price, or a drawing you update. `varip` also keeps values between ticks of the newest bar; most scripts never need it.

Exercises

1. **Reset each day.** Make the counter go back to 0 at the start of every day (hint: dayofmonth != dayofmonth[1]).
2. **Track the highest close.** Keep the highest close so far in a var float and plot it.

Lesson 7. Inputs

Make your script tunable with `input.*`. Each call adds a control to the settings dialog.

```
//@zscript 1
indicator("Tunable average", overlay=true)
len = input.int(20, "Length", minval=2, maxval=500)
src = input.source(close, "Source")
plot(ta.sma(src, len))
```

Give each input a plain title and a sensible default, set a min and max so nobody can type 0 as a length, and use `group=` to keep long lists tidy.

Exercises

1. **Three inputs.** Add a length, a color and a show-labels switch to one of your scripts.
2. **Pick list.** Use input.string with options to let the user choose between SMA and EMA, then switch on it.

Lesson 8. Loops, arrays and functions

A `for` loop repeats. An **array** is a list. A **function** names a piece of logic.

```
//@zscript 1
indicator("Last 5 closes average")
var prices = array.new<float>()
array.push(prices, close)
if array.size(prices) > 5
    array.shift(prices)
plot(array.avg(prices))
```

```
mid(h, l) => (h + l) / 2
```

Loops run on every bar, so keep them short. A built-in such as `ta.highest` is almost always faster than the same loop. Zscript stops runaway loops with a clear error.

Exercises

1. **Write a function.** Write range(h, l) that returns high minus low as a percent of close, and plot it.
2. **Count in a loop.** Use for to count how many of the last 10 closes were above their open.

Lesson 9. Drawing: labels, lines, boxes and tables

Plots are one value per bar. **Drawings** are objects you create and change: `label.new`, `line.new`, `box.new`, `table.new`.

```
//@zscript 1
indicator("Pivots", overlay=true)
ph = ta.pivothigh(high, 5, 5)
if not na(ph)
    label.new(bar_index - 5, ph, "H", style=label.style_label_down)
```

A pivot needs bars on both sides to be confirmed, so it is known five bars late. Placing the label at `bar_index - 5` puts it where the pivot really was, and never reads a future bar.

Each kind has a maximum on screen. The oldest are removed when you exceed it, so delete what you no longer need.

Exercises

1. **Pivot lows.** Add green labels at pivot lows.
2. **Last-bar table.** Show ATR and RSI in a table written only on barstate.islast.

Lesson 10. Our data: order book, order flow, GEX and options

Zscript can read the extra data TradeZoro charts show. `book.*` is the order book, `flow.*` is order flow, `gex.*` is GEX levels and `opt.*` is ZoroOptions numbers.

```
//@zscript 1
indicator("Delta and walls", overlay=false)
plot(flow.delta, "Delta", style=plot.style_columns)
plot(gex.callwall, "Call wall")
```

These follow your plan like the charts do. If your plan does not include a kind of data, the call returns **na** and you see one short notice. Always protect your math with `nz()`.

Every value is the one known at that bar: no peeking ahead. GEX and options numbers are updates stored a few minutes at a time, not an instant feed.

Exercises

1. **Delta bars.** Plot flow.delta as columns, green above zero and red below.
2. **Two kinds.** Plot gex.callwall and opt.range("expected") on one chart. What shows if one is missing for your plan?

Lesson 11. Alerts and request.security

`alertcondition(condition, title, message)` declares a rule that shows in the chart's alert dialog.

```
alertcondition(ta.crossover(close, ta.sma(close, 50)), "Above 50",
"Close crossed above the 50 SMA")
```

`request.security(symbol, timeframe, expression)` reads a value from another timeframe or symbol. Add `[1]` to use the last *finished* higher-timeframe bar, so the value never changes under your feet.

Exercises

1. **Daily levels.** Plot the previous day's high and low on an intraday chart.
2. **Closed-bar alert.** Fire alert() only when barstate.isconfirmed is true.

Lesson 12. Limits, errors and publishing

Scripts from other members run in your browser, inside a locked room with no network, no storage and no access to the page. They also have work limits: the first run must finish in 20 seconds, loops have a turn cap, and arrays and drawings have size caps.

Errors show a line and column, say what is wrong in plain English, and often suggest a fix ("Did you mean ta.sma?").

When your script is ready, **Publish** it to the Script Library. Choose open source (people can read it) or protected (people can use it but not read it), and free or paid. Publishing is free. Say honestly what the script does and never claim results.

Education only. Not financial advice.

Exercises

1. **Break it on purpose.** Misspell ta.sma as ta.smaa and read the message. Fix it using the hint.
2. **Describe your script.** Write a 3-sentence description for the script you are proudest of, with no claims about results.